

RTL Design Sherpa

DDR4 / LPDDR4 Family Controller Micro-Architecture Specification 0.5

October 3, 2026

Table of Contents

1 Andesite Mas Index....11
2 Document Information....12
2.1 andesite DDR4/LPDDR4 Family Controller Micro-Architecture Specification 12
2.2 Revision History....13
2.3 Document Purpose....18
2.4 Intended Audience....19
2.5 Related Documents....20
3 Block Inventory....21
3.1 Referenced, not rewritten: the inherited set....23
4 What Changes vs scoria....25
4.1 One paragraph per chapter page....27
5 Command Formatter (andesite_dfi_cmd_formatter)....29
5.1 Purpose....30
5.2 Parameters....31
5.3 Interface....32
5.4 Microarchitecture internals....34
5.4.1 DDR4 command truth table....34
5.4.2 Activate form....34
5.4.3 CA parity....35
5.4.4 NEW LPDDR4 CA submodule....35
5.4.5 Inline CKE and per-command CA cycles (LPDDR4)....35
5.5 FSM policy....37

5.6 Timing....38
5.7 Notes....39
5.8 P1 RTL interface (landed 2026-10-04)....40
6 Init Sequencer (andesite_init_sequencer)....41
6.1 Purpose....42
6.2 Parameters....43
6.3 Interface....44
6.4 Microarchitecture internals....49
6.4.1 DDR4 FSM state sequence....49
6.4.2 LPDDR4 FSM state sequence....49
6.4.3 Sequencing rules....49
6.5 FSM policy....51
6.6 Timing....52
6.7 Notes....53
6.8 CA parity error recovery....54
6.9 Gear-down programming (P1 interpretation, 2026-10-04)....55
7 Mode Register (andesite_mode_register)....56
7.1 Purpose....57
7.2 Parameters....58
7.3 Interface....59
7.4 Microarchitecture internals....61
7.4.1 DDR4 MR semantics....61
7.4.2 LPDDR4 MR set....62
7.4.3 MR coupling table....62

7.5 FSM policy..... 63
7.6 Timing..... 64
7.7 Notes..... 65
8 Address Mapper (andesite_addr_mapper)..... 66
8.1 Purpose..... 67
8.2 Parameters..... 68
8.3 Interface..... 69
8.4 Microarchitecture internals..... 70
8.4.1 Decode structure..... 70
8.4.2 Scheduler-visible outputs..... 70
8.4.3 Hashing..... 70
8.5 FSM policy..... 71
8.6 Timing..... 72
8.7 Notes..... 73
9 Scheduler and Command Arbiter (andesite_scheduler_plane + andesite_cmd_arbiter)..... 74
9.1 Purpose..... 75
9.2 Parameters..... 76
9.3 Interface..... 77
9.3.1 Host-side command ports..... 77
9.3.2 Bank-group timing ports..... 78
9.3.3 Maintenance request/grant ports..... 80
9.4 Macro init/policy interface (P1 rewiring, landed 2026-10-05)..... 81
9.5 Microarchitecture internals..... 83

9.5.1 FR-FCFS admission with L/S awareness....83
9.5.2 Global timers....83
9.5.3 Maintenance request/grant semantics....83
9.5.4 LPDDR4 degeneration....84
9.6 FSM policy....85
9.7 Timing....86
9.8 Notes....87
9.9 cmd_history_checker's DDR4 growth (landed 2026-10-04)....88
10 Refresh Controller (andesite_refresh_ctrl)....89
10.1 Purpose....90
10.2 Parameters....91
10.3 Interface....92
10.4 Microarchitecture internals....96
10.4.1 Inherited mechanism....96
10.4.2 LPDDR4 delta: controller-directed per-bank refresh....96
10.5 FSM policy....98
10.6 Timing....99
10.7 Notes....100
11 ZQ Calibration Controller (andesite_zq_ctrl)....101
11.1 Purpose....102
11.2 Parameters....103
11.3 Interface....104
11.3.1 Common ports (the landed DDR4 core)....104
11.3.2 LPDDR4-only ports (the landed MPC submodule)....106

11.4 Microarchitecture internals....108
11.4.1 DDR4 path: inherited from scoria....108
11.4.2 LPDDR4 path: new MPC submodule....108
11.4.3 Scheduler interface....109
11.5 FSM policy....110
11.6 Timing....111
11.7 Notes....112
12 ODT Controller (andesite_odt_ctrl)....113
12.1 Purpose....114
12.2 Parameters....115
12.3 Interface....117
12.4 Microarchitecture internals....119
12.4.1 Policy state table....119
12.4.2 Command-stream tap....119
12.4.3 Pin timing....119
12.4.4 The init seam....119
12.4.5 LPDDR4 delta....120
12.4.6 Telemetry....120
12.5 FSM policy....121
12.6 Timing....122
12.7 Notes....123
13 Training Interfaces (andesite_wrlvl_ifc, andesite_rdlvl_ifc, andesite_ca_train_ifc)....124
13.1 Purpose....125

13.1.1 The three family rules....125
13.2 Parameters....126
13.3 Interface....127
13.3.1 DFI training signal groups....127
13.3.2 CSR / host side....128
13.4 Microarchitecture internals....132
13.4.1 wrlvl_ifc: write leveling, MODIFIED....132
13.4.2 rdlvl_ifc: MPR read leveling, NEW....132
13.4.3 ca_train_ifc: LPDDR4 CA/WDQ training, NEW....132
13.4.4 Four-state telemetry....133
13.5 FSM policy....134
13.6 Timing....135
13.7 Notes....136
14 DFI Datapath (andesite_dfi_cmd_path, andesite_dfi_rd_aligner, andesite_dfi_wr_serializer, andesite_dfi_plane)....137
14.1 Purpose....138
14.2 Parameters....139
14.3 Interface....141
14.4 Microarchitecture internals....142
14.4.1 dfi_cmd_path: widened command bus....142
14.4.2 dfi_rd_aligner: DBI on the read path....142
14.4.3 dfi_wr_serializer: DBI on the write path....142
14.4.4 dfi_plane: the 4.0 control surface....142
14.4.5 LPDDR4 delta....143

14.5 FSM policy....144
14.6 Timing....145
14.7 Notes....146
15 DFI 4.0 Pin-Level Table....147
15.1 Command and address pins....148
15.2 Gear-down....152
15.3 Training and leveling pins....153
15.4 Data-path pins (DBI and the inherited surface)....156
15.5 Study corrections (2026-10-04, andesite TASK-005)....160
15.6 What this table does not do....161
16 Core Signal Contracts....162
16.1 What a contract is here....162
16.2 Pre-RTL citation posture....163
16.3 Contract: DDR4 command decode (the formatter’s core)....165
16.4 Contract: maintenance request/grant....166
16.5 Contract: training four-state telemetry....167
16.6 Regenerating the workbook....168

List of Figures

No figures in this document.

List of Tables

Table 1: Table 0.1: Related documents....20
Table 2: Table 1.1: MAS block inventory — the changed and new blocks, one page each....21
Table 3: Table 1.2: The inheritance list — referenced to scoria's books, with the where and the why....23
Table 4: Table 1.3: The marking set, copied from the HAS....25
Table 5: Table 2.1: Command formatter parameters....31
Table 6: Table 2.2: Command formatter ports....32
Table 7: Table 2.2.1: Init sequencer build parameters....43
Table 8: Table 2.2.2: Init sequencer ports....44
Table 9: Table 2.5: Mode register parameters....58
Table 10: Table 2.6: Mode register ports....59
Table 11: Table 2.7: DDR4 MR0-MR6 field semantics....61
Table 12: Table 2.8: Mode-register coupling....62
Table 13: Table 2.9: Address mapper parameters....68
Table 14: Table 2.10: Address mapper ports....69
Table 15: Table 2.5.1: Scheduler parameters....76
Table 16: Table 2.6.1: Refresh controller parameters....91
Table 17: Table 2.6.2: Refresh controller ports....92
Table 18: Table 2.7.1: ZQ controller parameters....103
Table 19: Table 2.7.2: ZQ controller ports....106
Table 20: Table 2.8.1: odt_ctrl parameters....115
Table 21: Table 2.8.2: odt_ctrl ports....117
Table 22: Table 2.9.1: Training-interface parameters....126
Table 23: Table 2.10.1: DFI datapath parameters....139
Table 24: Table 2.10.2: DFI datapath signal groups....141
Table 25: Table 3.1: DFI 4.0 command and address pins....148
Table 26: Table 3.2: Gear-down entry....152
Table 27: Table 3.3: Training and leveling pins....153
Table 28: Table 3.4: Data-path and inherited pins....156
Table 29: Table 4.1: Anchor map — the kmap generator cites these file:line pairs 163

1 Andesite Mas Index

Generated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 andesite DDR4/LPDDR4 Family Controller Micro-Architecture Specification

Property	Value
Document Title	andesite DDR4/LPDDR4 Family Controller Micro-Architecture Specification
Version	0.6 (draft)
Date	October 5, 2026
Status	v0.5 with the integration pass landed (2026-10-05): the scheduler macro carries the P1 rewiring, `dfi_cmd_path` exists, and the training/DFI/AXI4 plane assemblies plus `andesite_core` are built and suite-green (248 tests at the DFI 4.0 boundary). This book expands the owner-reviewed HAS's changed and new blocks to signal level. Every claim is inherited from a named scoria source, cited to JESD79-4 / JESD209-4 / DFI 4.0 with the `§TBC(TASK-005)` discipline, or recorded as an open question in the HAS ch06
Classification	Open Source - MIT License

2.2 Revision History

Version	Date	Author	Description
0.5	2026-10-05	RTL Design Sherpa	TASK-016 integration pass, signal level. Scheduler macro: P1 rewiring (DDR4-only init, `cmd_ack` = valid&&ready qualified on MRS/ZQCL, `zq_cal_start_o`, retired `dfi_init_*`/`t_*_wait_i`/`mr_wr_o` ports); command stream widened to `{ap,col,row,bg,bank,rank,op}` with `cmd_bg_o` (`CMD_W` grew by `$clog2(NUM_BG)`); the init request's duplicate-accept fix is a combinational gate on the accept cycle. `dfi_cmd_path` (new): A10 case-split address routing (RD/RDA/WR/WRA merge `ap<<10`, PREA forces A10, PRE clears, MRS/ZQCL keep the row-field payload). `andesite_training_plane` (new): the three carried training interfaces behind a maintenance command channel mirroring the init-source ack-pulse pattern. `andesite_dfi_plane` (new): CDC word packings `WD_DW={last,strb,dbi,data}` and `RD_DW={last,resp,dbi,data}` so write-DBI and read-

Version	Date	Author	Description
			DBI cross registered with their data; `dfi_init_complete_i` drives `pinit_complete_i`. `andesite_axi4_plane` (carried-modified): scoria rename-carry, byte-parity + logic-parity gates green. `andesite_core` (new top): four-plane wiring per the ch03 wiring table. Suites: fub 225, macro 19, core top 4 (248 total); master lint 53 modules; registry PASS. DV-framework drift recorded: the BFM behavior dispatch samples `cs_n`/error/init/update unconditionally — the init-slice TB carries `phy_dfi_cs_n` + idle handshake pins.
0.1	2026-10-03	RTL Design Sherpa	First draft, written before any RTL exists. Carries the HAS's changed/new blocks one level down: per-block interface tables, FSM policy, mechanism detail, and the contract anchors the generated kmap book cites. Inherited-unchanged blocks are referenced to scoria's books, not rewritten.
0.4	2026-10-04	RTL Design Sherpa	P3 gate: the modified/new tier is landed. 30 design

Version	Date	Author	Description
			files: 15 carried-inherited rename-only (13 active + the `powerdown_ctrl`/`dfi_signal_pack` dormant pair), 8 carried-modified (`addr_mapper` +bg, `global_timers` +L/S, `cmd_arbiter` +L/S admission, `refresh_ctrl` +FGR, `zq_ctrl` +MPC submodule, `mem_cmd_scheduler` carrying the FGR/MPC/recovery consumer paths, `dfi_wr_serializer`/`dfi_rd_aligner` +DBI), 4 new (`odt_ctrl`, `rdlvl_ifc`, `ca_train_ifc`, `zq_mpc_lpddr4`), and the 3 P1 blocks (`dfi_cmd_formatter`, `init_sequencer` +TASK-006 recovery FSM, `mode_register`). The marking-pair gate (HAS reuse table vs this book's What-Changes table) is GREEN at 23=23. The kmap CITES for the address-decode and MR-semantics fences now point at RTL anchor blocks in `addr_mapper`/`mode_register` (drift gate live, byte-identical reruns).

Version	Date	Author	Description
			`dfi_cmd_path` + `dfi_plane` remain per the plan: cmd_path rides the T9 integration pass; the plane is P4. The scheduler macro's init/mode_register rewiring and its macro-level suite land with that same pass.
0.3	2026-10-04	RTL Design Sherpa	P2 carry reconciliation: the inheritance table's ten active blocks exist as `andesite_*` with scoria's port lists (carry-diff gates prove rename/import/header-only deltas); `cmd_history_checker` carries the DDR4 L/S growth named on the scheduler page. The dormant pair exists uninstantiated. `rd_intake`/`wr_intake` land with P3's `addr_mapper` (they instantiate it).
0.2	2026-10-04	RTL Design Sherpa	P1 RTL reconciliation: Tables 2.2/2.2.2 (formatter, init_sequencer) and the mode_register interface now match the landed modules — v4.0 `dfi_cs` naming, 5-bit `op_i` carrying `OP_MPC`, 3-bit MR index on `cmd_bank`, 6-bit MR addresses, `csr_mrN_image`

Version	Date	Author	Description
			init payloads. Deferred ports (`dfi_alert_n`, `ca_o`/`ca_valid_o`) are recorded as follow-ons (TASK-006 / LPDDR4 breadth), not stubs. The kmap book's command-decode citations re-pointed at the RTL (drift gate live); per-command boolean SOP verdicts land with P3.
0.6	2026-10-20	RTL Design Sherpa	Section (h2) headings now open a new page, the pipeline default. No technical content changed.

2.3 Document Purpose

This Micro-Architecture Specification (MAS) is the implementation-level view of the andesite controller — the HAS (`../andesite_has/`) taken one level down. It is the document an RTL author reads before writing SystemVerilog and a verification engineer reads before writing checkers.

It covers:

- each changed or new block's interface, parameters, and internal mechanism
- the FSM policy for every block: where a state machine is required, where one is forbidden (no training search loops, per the D2 precedent), and what replaces FSMs elsewhere
- the DFI 4.0 pin-level table the HAS's interface chapter promises
- the core signal contracts whose anchors the generated kmap book (`../kmaps/`) verifies by citation

2.4 Intended Audience

- RTL implementers writing the changed and new blocks
- Verification engineers building the DFI 4.0 BFM checks and the formal spacing properties the HAS ch06 names
- Architects confirming that a block-level decision matches the HAS's marking and cause

2.5 Related Documents

Table 0.1: Related documents

Document	Location	Content
Hardware Architecture Specification	`../andesite_has/andesite_has_index.md`	the owner-reviewed v0.1 architecture this document expands; bindings between the two follow the HAS
Bootstrap design spec	`../../../../../../docs/superpowers/specs/2026-10-03-andesite-bootstrap-design.md`	the tranche's scope decisions (§6 is this book's plan)
Family docs	`../../../../docs/`	shared-core design (`mem_ctrl_pkg`) and family doctrine
scoria HAS	`../../../scoria-ddr3-lpddr3/docs/scoria_has/`	the reuse pool; authoritative for inherited-unchanged blocks
scoria design requirements	`../../../scoria-ddr3-lpddr3/docs/design-requirements.md`	the delta-analysis method; TASK-001 mode detail (§6)
Kmap generator + workbook	`../kmaps/`	generated command-encoding tables citing this book's anchors
Family README	`../../../../../../README.md`	family overview and per-IP status

3 Block Inventory

The MAS documents the changed and new blocks below, one page per chapter in `ch02_blocks/`. Each page follows the house FUB-page skeleton: header metadata, purpose, parameters, interface, microarchitecture internals, FSM policy, timing, and notes — the skeleton the bch MAS established and the pumice MAS before it. Markings are copied from the HAS, not re-derived; a marking here that disagrees with the HAS is a defect here.

Table 1.1: MAS block inventory — the changed and new blocks, one page each

Block	Marking	MAS page	HAS pointer
`dfi_cmd_formatter` (+ NEW LPDDR4 CA submodule)	MODIFIED / NEW	ch02_blocks/01_cmd_formatter.md	Ch 3.2, 3.6
`init_sequencer`	MODIFIED	ch02_blocks/02_init_sequencer.md	Ch 3.2
`mode_register`	MODIFIED	ch02_blocks/03_mode_register.md	Ch 3.2
`addr_mapper`	MODIFIED	ch02_blocks/04_addr_mapper.md	Ch 3.1
`scheduler_plane` + `cmd_arbiter`	MODIFIED	ch02_blocks/05_scheduler.md	Ch 3.1
`refresh_ctrl`	MODIFIED	ch02_blocks/06_refresh_ctrl.md	Ch 3.4
`zq_ctrl` (+ NEW LPDDR4 MPC submodule)	INHERITED / NEW	ch02_blocks/07_zq_ctrl.md	Ch 3.2
`odt_ctrl`	NEW	ch02_blocks/08_odt_ctrl.md	Ch 3.5
training interfaces (`wrlvl_ifc`, `rdlvl_ifc`, `ca_train_ifc`)	MODIFIED / NEW	ch02_blocks/09_training.md	Ch 3.3

Block	Marking	MAS page	HAS pointer
DFI datapath (`dfi_cmd_path`, `dfi_rd_aligner`, `dfi_wr_serializer`, `dfi_plane`)	MODIFIED	ch02_blocks/10_dfi_datapath.md	Ch 3.5, 4.1

Ten pages, matching the ten areas the HAS's Chapter 3 argues: the command encoding, init, mode registers, address mapping, scheduling, refresh, ZQ, ODT, training, and the DFI datapath. The kmap book's citations point into these pages' mechanism sections; where a page names an encoding or a decode equation, that line is an anchor.

3.1 Referenced, not rewritten: the inherited set

The inherited-unchanged blocks are not MAS pages. Each is specified by scoria's book and its RTL header, and this MAS states where and why:

Table 1.2: The inheritance list — referenced to scoria's books, with the where and the why

scoria module	Marking	Where its specification lives	Why not rewritten here
`scoria_top`, `scoria_top_geared`, `scoria_core`, `scoria_axi4_ifc`	INHERITED	scoria HAS Ch 2.3 / 4.2 and the `rtl/top/` + `rtl/macro/` headers	structure and host side unchanged; andesite's deltas do not reach them
`bank_timer` / `bank_timers`, `page_policy`, `rd_cmd_cam`, `wr_data_cam`	INHERITED	scoria HAS Ch 2.3 and the `rtl/fub/` headers	the scheduling *mechanism* is unchanged; only what the arbiter asks of it (the L/S pairs) is new, and that belongs to the scheduler page
`rd_intake`, `wr_intake`, `wr_splitter`, `rd_return_ring`, `axi_burst_chopper`	INHERITED	scoria HAS Ch 2.3 / 4.2	the host-side front end is untouched by this generation
`dfi_cdc`	INHERITED	scoria HAS Ch 2.3 / 4.1	the CDC shape transfers to DFI 4.0 unchanged (HAS Ch 4.1)
`cmd_history_checker`	INHERITED	scoria HAS Ch 2.3	mechanism inherited; its DDR4 spacing-parameter growth is named on the scheduler page,

scoria module	Marking	Where its specification lives	Why not rewritten here
			where the parameters are defined
`powerdown_ctrl`, `dfi_signal_pack`	DORMANT	andesite HAS Ch 3.1	carried dormant with the waking condition named there; a dormant block has no mechanism news to specify

The rule both tables enforce: the MAS expands the HAS's changed/new blocks; it does not re-derive, re-mark, or silently restate anything the HAS and scoria's books already settle.

4 What Changes vs scoria

The complete marking set, copied from the HAS (Ch 2.3 and Table 3.1) rather than re-derived. This table and the HAS carry the same pairs; the Task-10 consistency gate diffs them mechanically.

Table 1.3: The marking set, copied from the HAS

scoria module	Marking	What changes, one line
`scoria_top`	INHERITED	structure unchanged
`scoria_top_geared`	INHERITED	the width-gearing wrapper, unchanged
`scoria_core`	INHERITED	core assembly unchanged
`scoria_axi4_ifc`	INHERITED	the AXI4 host side is untouched by this generation
`scoria_mem_cmd_scheduler`	MODIFIED	L/S admission alongside scoria's ZQ maintenance admission
`scoria_dfi_plane`	MODIFIED	DFI 4.0 control surface and DBI wires
`scoria_addr_mapper`	MODIFIED	bank-group decode: BG0/BG1 between chip select, bank and row
`scoria_cmd_arbiter`	MODIFIED	L/S-aware issue on the same-group / cross-group split
`scoria_global_timers`	MODIFIED	tCCD_L/S and tRRD_L/S as first-class counters
`scoria_dfi_cmd_formatter`	MODIFIED	ACT_n fifth pin, BG0/BG1, CA parity; NEW submodule: LPDDR4 6-bit CA path
`scoria_init_sequencer`	MODIFIED	DDR4 reset procedure, MR3-MR6-MR5-MR4-MR2-MR1-MR0 order, gear-down entry, parity

scoria module	Marking	What changes, one line
		enable
scoria_mode_register	MODIFIED	MR0-MR6 field maps per memtype
scoria_zq_ctrl	INHERITED	DDR4's ZQCS/ZQCL exactly as built; NEW submodule: LPDDR4 MPC calibration
scoria_refresh_ctrl	MODIFIED	FGR 1x/2x/4x on the inherited elastic/TCR/placement base
scoria_dfi_cmd_path	MODIFIED	ACT_n, BG and parity wires ride the command path
scoria_dfi_rd_aligner	MODIFIED	DBI read path
scoria_dfi_wr_serializer	MODIFIED	DBI write path
scoria_wrlvl_ifc	MODIFIED	training-interface family grows; contract and no-search rule unchanged
scoria_powerdown_ctrl	DORMANT	carried dormant, waking condition in HAS Ch 3.1
scoria_dfi_signal_pack	DORMANT	carried dormant, waking condition in HAS Ch 3.1
(new) odt_ctrl	NEW	dynamic ODT: RTT_NOM/WR/PARK and the ODT latency family
(new) rdlvl_ifc	NEW	MPR-based read-leveling interface, no search loop
(new) ca_train_ifc	NEW	LPDDR4 CA/WDQ training interface, no search loop

4.1 One paragraph per chapter page

Command formatter (MODIFIED / NEW). The wire-level truth table changes shape: ACT becomes a five-pin command, BG0/BG1 appear on the activate's address pins, and parity rides the command stream. The LPDDR4 side is a new 6-bit double-data-rate CA encoding, two cycles per command — a submodule so the DDR4 path stays reviewable on its own. Mechanism detail: Ch 2.1.

Init sequencer (MODIFIED). The FSM grows DDR4's reset-procedure states and gear-down entry, and programs parity enable in the MR order. The order remains JEDEC's, now MR3-MR6-MR5-MR4-MR2-MR1-MR0; the MAS page lists the states and the waits by name, with values per the HAS's Q1 recording. Mechanism detail: Ch 2.2.

Mode register (MODIFIED). MR0-MR6 for DDR4 and the LPDDR4 MR set, field maps per memtype, with the MAS page carrying the field-level tables the HAS deliberately leaves to this book. Mechanism detail: Ch 2.3.

Address mapper (MODIFIED). One bounded change: the decode grows BG0/BG1 between chip select, bank and row. The decode equations are the kmap book's first address-map targets. Mechanism detail: Ch 2.4.

Scheduler and arbiter (MODIFIED). L/S-aware issue: the arbiter consults same-group versus cross-group spacing, the global timers carry the long/short pairs, and maintenance admission (refresh, ZQ, ODT turnarounds) rides the inherited request/grant channel. LPDDR4 degenerates L = S gracefully. Mechanism detail: Ch 2.5.

Refresh controller (MODIFIED). FGR 1x/2x/4x via the MR3 image, interval arithmetic scaled per factor, and LPDDR4's controller-named per-bank scheduling replacing the rotor. The elastic/TCR/placement policy base is inherited intact. Mechanism detail: Ch 2.6.

ZQ controller (INHERITED / NEW). The DDR4 core is scoria's, unchanged. The new submodule sequences LPDDR4 calibration through MPC. Mechanism detail: Ch 2.7.

ODT controller (NEW). The policy block: RTT_NOM/WR/PARK selection on the command-stream tap, ODT pin latency enforcement, rank coupling for multi-rank builds. LPDDR4's MR-programmed termination is init-side, not this page's. Mechanism detail: Ch 2.8.

Training interfaces (MODIFIED / NEW). `wrlvl_ifc` keeps its contract; `rdlvl_ifc` and `ca_train_ifc` are new pages in the same family — handshake, MR/MPC path, windows, four-state telemetry, no search loops anywhere. Mechanism detail: Ch 2.9.

DFI datapath (MODIFIED). `dfi_cmd_path` widens for ACT_n/BG/parity; the aligner and serializer pass DBI through; the plane presents the 4.0 control surface. The CDC and the pack stage are inherited (the latter dormant). Mechanism detail: Ch 2.10.

5 Command Formatter (andesite_dfi_cmd_formatter)

Module: andesite_dfi_cmd_formatter.sv **Location:** rtl/fub/ (planned) **Category:** FUB
Parent: dfi_cmd_path / dfi_plane **Status:** specified — no RTL exists (HAS v0.1 posture)

5.1 Purpose

`dfi_cmd_formatter` is the last combinational step before the command stream hits the DFI pins. It takes the scheduler's abstract opcode and address fields and turns them into the wire pattern the DRAM expects: ACT_n, RAS_n, CAS_n, WE_n, chip-select, bank, bank-group and address for DDR4; the 6-bit double-data-rate CA bus for LPDDR4. The block doesn't sequence anything — it just encodes. Sequence lives in the scheduler and the init_sequencer, the same split pumice and scoria used.

The andesite delta is the DDR4 fifth pin and bank-group wiring, plus a NEW LPDDR4 CA submodule that sits beside the DDR4 path so each stays reviewable on its own. Parity generation also lands here, gated by init and returned on `dfi_alert_n`.

5.2 Parameters

Table 2.1: Command formatter parameters

Parameter	Type	Range	Default	Meaning	Source
ADDR_WIDTH	int	18	18	row/ column address width presented to formatter	design point
BANK_WIDTH	int	2	2	bank address width (DDR4); 3 for LPDDR4	design point
BG_WIDTH	int	2	2	bank-group width (DDR4 only)	design point
RANK_WIDTH	int	1	1	rank/chip-select width	design point
CA_WIDTH	int	6	6	LPDDR4 CA bus width	JESD209-4
PARITY_EN	bit	0/1	0	CA parity generation enable (dynamic after init)	CSR

5.3 Interface

Table 2.2: Command formatter ports

Signal	Direction	Width	Description
`clk`	in	1	controller clock; DFI 1:4 gear ratio means one command beat per four DRAM clocks §TBC(TASK-005)
`rst_n`	in	1	active-low synchronous reset
`op_i`	in	5	internal opcode, `andesite_pkg::dram_op_e`: `OP_NOP`..`OP_ZQCL` carried from scoria plus `OP_MPC`; the 5-bit enum carries the LPDDR4 MPC opcode selection
`bank_i`	in	`BANK_WIDTH`	bank address from the mapper
`bg_i`	in	`BG_WIDTH`	bank-group address from the mapper (DDR4)
`row_i`	in	`ADDR_WIDTH`	row address, valid with `OP_ACT`
`col_i`	in	`ADDR_WIDTH`	column address, valid with `OP_RD`/`OP_WR`
`rank_i`	in	`RANK_WIDTH`	chip select / rank target
`memtype_i`	in	3	family memtype

Signal	Direction	Width	Description
			enum from andesite_pkg
parity_en_i	in	1	enable CA parity generation after init
dfi_act_n	out	1	DFI 4.0 activate command pin §TBC(TASK-005)
dfi_ras_n	out	1	DFI row-address strobe command pin
dfi_cas_n	out	1	DFI column-address strobe command pin
dfi_we_n	out	1	DFI write-enable command pin
dfi_bank	out	BANK_WIDTH	DFI bank address
dfi_bg	out	BG_WIDTH	DFI bank-group address (DDR4)
dfi_address	out	ADDR_WIDTH	DFI address bus
dfi_cs	out	RANKS	DFI chip-select, active low; v4.0 catalog name (no *_cs_n)
dfi_cke	out	1	DFI clock-enable; DDR4-scoped (LPDDR4 carries CKE inline on the CA bus)
dfi_parity_in	out	1	DFI CA parity output to PHY §TBC(TASK-005)

5.4 Microarchitecture internals

5.4.1 DDR4 command truth table

The DDR4 encoding follows the JESD79-4 four-pin command convention, with ACT_n as the fifth pin for activate. The truth table is the citation anchor for the block; don't renumber it.

```
CS_n=0:
  ACT_n  RAS_n  CAS_n  WE_n    Command
    1      1      1      1     NOP
    0      1      1      1     ACT
    1      1      0      1     RD
    1      1      0      0     WR
    0      0      0      0     MRS
    0      0      0      1     REF
    1      0      1      0     PRE     (A10=0: PRE per bank; A10=1:
PREA)
    1      1      1      0     ZQ      (A10=0: ZQCS; A10=1: ZQCL)

RD/WR auto-precharge variants (RDA/WRA) reuse the RD/WR pin encoding
with
the AP address bit set (A10=1) -- the auto-precharge select is an
address
input, not a pin variant, so the kmap book treats AP as a table input.

CS_n=1: DES (deselect)

CKE-qualified entries: SRE (self-refresh entry), SRX (self-refresh
exit)
LPDDR4-only command: MPC (multipurpose command), see CA submodule
below
```

: Table 2.3: DDR4 command truth table (citation anchor). Pin encodings are stated per the JESD79-4 command-truth-table convention and are confirmed at the HAS Q1 cold-storage read before RTL; the kmap book's DDR4 table (andesite TASK-004) minimizes its qualifier SOPs from exactly this anchor.

A10 inside PRE selects precharge-all (PREA); A10 inside ZQ selects ZQCL versus ZQCS; A10 inside RD/WR selects the auto-precharge variants (RDA/WRA). During ACT, A10 is not a variant-select bit; it carries row address bit 10 like any other activate address pin. BG0 and BG1 ride the address bus during ACT.

5.4.2 Activate form

The activate command packs the row and bank-group fields onto the DFI address and bank-group outputs:

```
{ ACT_n=0, dfi_bank[1:0]=BA[1:0], dfi_bg[1:0]=BG[1:0],
dfi_address[17:0]=row[17:0] }
```

: Figure 2.1: DDR4 activate field packing

5.4.3 CA parity

Parity is generated in the formatter from the command pins plus the address/bank/bank-group bits that travel on the same DRAM clock edge. The insertion point is the registered pipeline stage that drives `dfi_parity_in`. Generation only runs when `parity_en_i` is asserted; that bit is cleared at reset and set by the init_sequencer once the MR sequence has programmed parity mode. This is the same init-side enable dependency scoria used for DLL-related gating.

A parity error returned on `dfi_alert_n` is a DFI 4.0 alert event (`§3.5.7` command-address parity, `§4.11` alert handling). The formatter logs the raw `dfi_alert_n` pulse; the sequencer owns the recovery policy and the MR re-programming path, described in `ch02_blocks/02_init_sequencer.md` §“CA parity error recovery”.

5.4.4 NEW LPDDR4 CA submodule

LPDDR4 uses a 6-bit double-data-rate CA bus, two cycles per command (RD/WR); ACT is issued as an ACT-1/ACT-2 pair. The submodule takes the internal opcode, bank, and MPC/MRW payload and emits the two-cycle CA beat pair. Each command is therefore a 12-bit symbol split across two half-rates. The exact CA encodings are pinned by the kmap book’s LPDDR4 CA-bus table in `docs/kmaps/generated/` (andesite TASK-004); this page does not fabricate them.

```
Command class -> CA encoding source
  NOP      -> kmap table
  ACT      -> kmap table
  RD       -> kmap table
  WR       -> kmap table
  MPC      -> kmap table
  MRW      -> kmap table
  MRR      -> kmap table
  REFab    -> kmap table
  REFpb    -> kmap table (bank carried in command)
```

: Table 2.4: LPDDR4 CA encoding placeholder (kmap book anchor)

The submodule is combinational encoding plus the two-cycle registered shifter. The first cycle drives `ca_o` with CA[5:0]; the second cycle drives `ca_o` with CA[11:6]. `ca_valid_o` marks both cycles.

5.4.5 Inline CKE and per-command CA cycles (LPDDR4)

LPDDR4 removed the dedicated CKE pin — JESD209-4E carries the CKE-equivalent state inline on the CA bus, and the LPDDR4 research index lists this as one of the big architectural breaks from LPDDR3. The `dfi_cke` output of this formatter is therefore DDR4-scoped only; for LPDDR4, the CA submodule encodes CKE-equivalent states (power-down entry/exit, self-refresh entry/exit) as CA-

bus transactions. That ties directly to the HAS Ch 3.1 dormant-pair story: `powerdown_ctrl` and `dfi_signal_pack` are DDR4-scoped this edition, while LPDDR4 low-power states live in the CA encoding.

The per-command CA cycle structure also differs from the flat “two cycles per command” wording in the purpose section. RD and WR span two CA cycles, but ACT is not a single two-cycle command — it is issued as an ACT-1 / ACT-2 pair on two separate clocks. The bit-level CA layouts for every command class are per JESD209-4E cold-storage confirmation; the kmap book’s LPDDR4 CA table (`docs/kmaps/generated/02_lpddr4_ca_command_table.md`) is the authoritative placeholder and stays TBC until that confirmation is done.

5.5 FSM policy

There is no FSM. The formatter is combinational encoding followed by a registered pipeline stage; the LPDDR4 submodule adds a two-cycle shift register for the second CA beat. This is the pumice/scoria precedent: encode here, sequence elsewhere.

5.6 Timing

All command-to-command spacing is enforced upstream by `global_timers` and the scheduler. The formatter introduces one controller-clock pipeline delay from decoded inputs to DFI outputs. LPDDR4 commands consume two controller clocks on the CA bus, so the downstream command accept handshake reflects that stretch. Specific JEDEC values (`tCCD_L`, `tCCD_S`, `tMRD`, `tMOD`, the ODT latency family) are runtime CSRs, initialised from the JESD79-4/JESD209-4 speed bin at CSR-derivation time (HAS Ch 5; numeric constants are HAS open question Q1).

5.7 Notes

- Keep the DDR4 and LPDDR4 paths separate at the top level of this FUB. Merging them makes review harder and gains nothing — the memtype select is one mux at the output boundary.
- `dfi_alert_n` is asynchronous at the PHY; sample it synchronously inside this block and export a one-cycle pulse.
- LPDDR4's REFpb command names the bank explicitly in the CA encoding, which is why the refresh controller's per-bank scheduling lands naturally on this path.

5.8 P1 RTL interface (landed 2026-10-04)

Table 2.2 above now matches the landed `andesite_dfi_cmd_formatter`: v4.0 `dfi_cs` naming (no `*_cs_n`), a 5-bit `op_i` carrying `OP_MPC` in the enum (the standalone `mpc_op_i` port was retired with it), and a scalar `dfi_cke` (DDR4-scoped; LPDDR4's inline CKE rides the CA bus). The `dfi_alert_n` input and the `ca_o`/`ca_valid_o` pair are deferred on purpose — they land with the andesite TASK-006 recovery FSM and the LPDDR4 breadth track respectively, not as untested stubs. The `CA_WIDTH` and `PARITY_EN` rows of Table 2.1 are design-point parameters; the RTL carries them as the CA-bus width constant and the `parity_en_i` port.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 Init Sequencer (andesite_init_sequencer)

Module: andesite_init_sequencer.sv **Location:** rtl/init/ **Category:** control / initialization
Parent: andesite_core / andesite_top **Status:** specified — no RTL exists (HAS v0.1 posture)

6.1 Purpose

`init_sequencer` owns the power-up handshake with the DRAM. It drives RESET#, raises CKE, programs the mode-register set in JEDEC order, issues ZQCL, waits out tDLLK and tZQinit, and optionally performs the gear-down entry sequence. Until it reaches READY, the sequencer holds the command bus: no host traffic reaches the formatter. After READY, it goes idle but stays available for a firmware-triggered re-initialization.

The block is MODIFIED from scoria because DDR4 adds the RESET# pin, seven mode registers, gear-down, and CA parity enable; LPDDR4 needs its own bus-reset, MRW order, and MPC-carried calibration handoff.

6.2 Parameters

Table 2.2.1: Init sequencer build parameters

Parameter	Type	Range	Default	Meaning
MEMTYPE	memtype_e	DDR4, LPDDR4	build	selects the init branch and the MR set
NUM_CHANNELS	int	1..2	1	LPDDR4 channel count; DDR4 uses 1
GEARDOWN_SUPPORTED	bit	0,1	1	when 1, the FSM includes the gear-down entry states

All timing values are runtime CSRs, not parameters. That's the family rule: a timing compiled in cannot be swept, and a controller that cannot be swept cannot be characterized.

6.3 Interface

Table 2.2.2: Init sequencer ports

Signal	Direction	Width	Description
clk	in	1	controller clock
reset_n	in	1	active-low reset
csr_init_trigger	in	1	firmware pulse: drop back to POWER_ON_RESET and re-run
csr_memtype	in	3	runtime copy of MEMTYPE for the FSM branch
csr_geardown_en	in	1	enable gear-down entry after ZQCL
csr_parity_en	in	1	enable CA parity through the MR sequence
tinit1_csr	in	TINIT_WIDTH	tINIT1 wait, runtime CSR
tinit3_csr	in	TINIT_WIDTH	tINIT3 wait, runtime CSR
tinit4_csr	in	TINIT_WIDTH	tINIT4 wait, runtime CSR
tdllk_csr	in	TINIT_WIDTH	tDLLK wait, runtime CSR
tzqinit_csr	in	TINIT_WIDTH	tZQinit wait, runtime CSR
tmrd_csr	in	TINIT_WIDTH	tMRD wait,

Signal	Direction	Width	Description
			runtime CSR
tmod_csr	in	TINIT_WIDTH	tMOD wait, runtime CSR
mr_image_out	out	packed MR vector	current MR value for mode_register
mr_load	out	1	pulse: mode_register captures mr_image_out
reset_n_out	out	1	RESET# pin to top/PHY, active low
cke_out	out	1	CKE pin, registered from activation until init complete
cmd_req	out	1	request to the formatter/scheduler for the next init command
cmd_ack	in	1	grant: the command is issued this cycle
cmd_op	out	opcode	init command opcode (MRS/MRW/ZQCL/MPC/etc.)
cmd_bank	out	3	bank address (2 bits used);

Signal	Direction	Width	Description
			the MRS path carries the 3-bit MR index (0-6)
cmd_addr	out	address width	MR index / MRW address field
zq_cal_start	out	1	pulse to zq_ctrl to start ZQ calibration
gear_down_entry	out	1	pulse to PHY/formatter to enter gear-down §TBC(TASK-005)
parity_enable_out	out	1	parity is now active; formatter begins counting
init_done	out	1	READY reached, normal operation may begin
init_err	out	1	sequencing or timeout error, latched until re-init
ca_train_start	out	1	LPDDR4 handoff to ca_train_ifc when training

Signal	Direction	Width	Description
			is required
parity_alert_i	in	1	the formatter's logged CA-parity alert pulse; the recovery FSM's entry event
recovery_interval_i	in	16	JEDEC-named recovery interval for RESENDING, runtime CSR
csr_telem_clear_i	in	1	explicit firmware clear of the recovery telemetry
retract_req_o	out	1	maintenance-class request asking the scheduler to withdraw the suspect command
retract_ack_i	in	1	scheduler grant for the retract; request-and-wait, like cmd_req/cmd_ack
obs_recovery_state_o	out	2	recovery sub-FSM state: 0

Signal	Direction	Width	Description
			IDLE, 1 ALERT_SEEN, 2 RESENDING
obs_alerts_seen_o	out	16	saturating telemetry: alerts seen
obs_cmds_dropped_o	out	16	saturating telemetry: suspect commands dropped
obs_cmds_resent_o	out	16	saturating telemetry: commands released for re-issue

The cmd_req/cmd_ack pair is the same request-and-wait discipline the scheduler uses for maintenance traffic. Init owns the bus, but it still waits for the formatter to be ready.

6.4 Microarchitecture internals

6.4.1 DDR4 FSM state sequence

The DDR4 branch is one flat FSM. Gear-down entry is folded as states, not a nested machine.

```
POWER_ON_RESET
    -> RESET_ASSERT        (drive RESET# low; wait tINIT1)
    -> RESET_DEASSERT_WAIT  (RESET# high; wait tINIT3)
    -> CKE_ENABLE           (raise CKE)
    -> CKE_WAIT            (wait tINIT4)
    -> MRS_MR3
    -> MRS_MR6
    -> MRS_MR5
    -> MRS_MR4
    -> MRS_MR2
    -> MRS_MR1
    -> MRS_MR0
    -> ZQCL
    -> DLLK_ZQINIT_WAIT    (wait tDLLK and tZQinit)
    -> [GEARDOWN_ENTRY if csr_geardown_en: program MR3 gear-down,
        issue entry sequence, wait sync]
    -> READY
```

6.4.2 LPDDR4 FSM state sequence

LPDDR4 uses its own branch. The exact MRW order follows JESD209-4; the table below captures the structural stages.

```
POWER_ON_RESET
    -> BUS_RESET            (reset over CA bus or reset pin per
JESD209-4)
    -> INIT_WAIT            (JESD209-4 initialization wait)
    -> MRW_SEQUENCE        (JESD209-4 order: device feature, ODT,
                            drive-strength, DQ-ODT, and remaining MRs)
    -> MPC_ZQ_CALIBRATION  (issue MPC ZQCal start / latch via
formatter CA path)
    -> [CA_TRAINING_HANDOFF if required: ca_train_start to
ca_train_ifc]
    -> READY
```

6.4.3 Sequencing rules

The MR order is a citation, not a design choice. pumice's EMRS3-first correction is the family precedent, and andesite follows it: MR3 first, then MR6, MR5, MR4, MR2, MR1, MR0 last. MR0 carries the DLL reset only; CA parity latency/mode is programmed in MR5, so parity enable rides the MR5 program step. MR0 must wait until the bus is stable.

Parity enable is ordered after bus stability for the same reason. Once `parity_enable_out` rises, the formatter's parity counter starts counting from that point; enabling it earlier would count commands that were not parity-protected.

`CKE` is continuously registered from the `CKE_ENABLE` state until init completes. Dropping it mid-sequence is an error and is flagged by `init_err`.

6.5 FSM policy

This block has **one FSM** and it is the init FSM. There are no nested FSMs and no sub-state machines hidden inside the MR sequence. Gear-down entry is a sequence of states in the same FSM; the PHY/formatter handshake is a single output pulse plus a wait state.

The MR states themselves are simple: load the next MR image, pulse `mr_load`, assert `cmd_req`, wait for `cmd_ack`, then wait `tMRD` or `tMOD` as required before the next state.

6.6 Timing

Every wait state counts from a CSR-loaded value. The values are runtime CSRs, initialised from the JESD79-4/JESD209-4 speed bin at CSR-derivation time (HAS Ch 5; numeric constants are HAS open question Q1).

Wait	Symbol	Loaded from	Counted in
Reset assertion	`tINIT1`	`tinit1_csr`	MC clocks
Reset deassert to CKE	`tINIT3`	`tinit3_csr`	MC clocks
CKE to first MRS/MRW	`tINIT4`	`tinit4_csr`	MC clocks
Between MRS commands	`tMRD`	`tmrd_csr`	MC clocks
MRS to non-MRS command	`tMOD`	`tmod_csr`	MC clocks
DLL lock + ZQ init	`tDLLK`, `tZQinit`	`tdllk_csr`, `tzqinit_csr`	MC clocks, both must expire

The counter reload value for each state is captured when the state is entered, so a CSR written mid-wait does not affect the running count. That's the same reload-only rule the refresh controller uses for `tREFI` derate.

Ordering checks — for example, that MR3 is issued before MR0 — are assertions in DV, not in RTL. That's the family rule: the FSM encodes the order, and DV proves the FSM never strays.

6.7 Notes

- **LPDDR4 calibration handoff:** `MPC_ZQ_CALIBRATION` does not perform the calibration itself. It issues the MPC opcodes through the formatter's LPDDR4 CA submodule and then raises `init_done`. If the board requires CA training, the FSM can hand off to `ca_train_ifc` before `READY`.
- **Re-initialization:** A firmware pulse on `csr_init_trigger` forces the FSM back to `POWER_ON_RESET`. The `init_done` output falls immediately so the scheduler stops admitting host commands.
- **Gear-down and parity are independent CSR selects.** Either, both, or neither may be enabled; the FSM states exist for both paths.
- **DFI 4.0 gear-down handshake:** The `gear_down_entry` pulse crosses to the PHY through the DFI 4.0 control surface `§TBC(TASK-005)`. The exact DFI signal names are public and live in the formatter chapter.

6.8 CA parity error recovery

Recovery from a CA parity alert is handled by a small sub-FSM that sits beside the init FSM and never enters the bank machine. The formatter logs the raw `dfi_alert_n` pulse (see `ch02_blocks/01_cmd_formatter.md` §"CA parity"); that logged pulse is the entry event for this recovery FSM. DFI 4.0 alert and parity semantics are `§3.5.7` and `§4.11`.

```
IDLE -> ALERT_SEEN -> RESENDING -> IDLE
```

- `IDLE` — waiting. The recovery FSM is transparent; normal scheduler grants pass through unchanged.
- `ALERT_SEEN` — on the cycle the formatter's logged pulse arrives, the command currently in the grant stream is marked suspect and dropped. The FSM asserts a `retract` sideband to the scheduler, asking it to withdraw the suspect command and re-issue it from its request queue. The scheduler's request-never-preempts rule is preserved: the recovery request is just another maintenance-class request that must wait for the scheduler's grant, exactly like the init sequencer's `cmd_req`/`cmd_ack` pair.
- `RESENDING` — the FSM holds the JEDEC-named recovery interval, counting down, then returns to `IDLE`. It never drives a command out of `RESENDING`: release-to-scheduler, not re-issue — the scheduler re-issues the dropped command from its own request queue once the retract clears. The interval value is a runtime CSR loaded from the JESD79-4 speed bin at CSR-derivation time; no numeric constant is compiled into the RTL.

Telemetry is kept in small saturating counters: alerts seen, commands dropped, and commands re-issued. These counters are visible to firmware and are reset only by controller reset or an explicit firmware clear.

The recovery FSM does **not** trigger a full re-initialization. A single parity event drops one command and retransmits it; only a sustained or uncorrectable pattern would escalate to firmware-assisted MR5 re-programming or init re-run. That escalation policy is recorded as open question Q4 in the HAS, not hard-wired here.

6.9 Gear-down programming (P1 interpretation, 2026-10-04)

The fence's "program MR3 gear-down" step rides the MR image: the FSM issues the normal MR3 MRS with whatever image firmware configured (`csr_mr3_image`), then the entry pulse and the sync wait follow ZQ-init. The gear-down bit's position in MR3 is Q1 (JESD79-4 cold-storage read) – the RTL makes no bit-position claim; the P1 test asserts the configured bit reaches the bus, wiring-only.

---

7 Mode Register (andesite_mode_register)

Module: andesite_mode_register.sv **Location:** rtl/fub/ (planned) **Category:** FUB **Parent:** init_sequencer / dfi_cmd_formatter / training interfaces **Status:** specified — no RTL exists (HAS v0.1 posture)

7.1 Purpose

`mode_register` holds the MR images per memtype and feeds them to the formatter in the order the init_sequencer demands. It also services runtime MR reads and writes — firmware training paths re-touch MR3 for MPR access and MR1 for write leveling, and the LPDDR4 MR set gets similar treatment during CA and WDQ training. The block is a set of registers plus a mux; it doesn't own the programming order. Order is the init_sequencer's property, and this page cites that split rather than duplicating it.

7.2 Parameters

Table 2.5: Mode register parameters

Parameter	Type	Range	Default	Meaning	Source
DDR4_MR_COUNT	int	7	7	MR0-MR6 for DDR4	JESD79-4
LPDDR4_MR_COUNT	int	TBD	TBD	LPDDR4 MR space; exact count per JESD209-4	Q1
DATA_WIDTH	int	16	16	MR payload width (max across both memtypes)	design point
RANK_WIDTH	int	1	1	rank select for per-rank MR images	design point

7.3 Interface

Table 2.6: Mode register ports

Signal	Direction	Width	Description
clk	in	1	controller clock
rst_n	in	1	active-low synchronous reset
memtype_i	in	3	family memtype enum from andesite_pkg
rank_i	in	RANK_WIDTH	rank target for MR access
wr_en_i	in	1	write strobe from init_sequencer or firmware
wr_addr_i	in	6	MR index (0-6 for DDR4; LPDDR4 index per JESD209-4, 64-entry working bound)
wr_data_i	in	DATA_WIDTH	MR write payload
rd_en_i	in	1	read strobe for firmware readback
rd_addr_i	in	6	MR index for readback
rd_data_o	out	DATA_WIDTH	MR readback data
mr_sel_o	out	6	MR index to formatter for MRS/MRW command
mr_data_o	out	DATA_WIDTH	MR payload to formatter for MRS/MRW

Signal	Direction	Width	Description
			command
mpr_page_o	out	2	MR3 MPR page select to rdlvl_ifc
fgr_factor_o	out	2	MR3 FGR 1x/2x/4x select to refresh_ctrl
rtt_nom_o	out	3	MR1 RTT_NOM value to odt_ctrl
rtt_wr_o	out	3	MR2 RTT_WR value to odt_ctrl
rtt_park_o	out	3	MR5 RTT_PARK value to odt_ctrl
rd_dbi_en_o	out	1	MR5 read DBI enable to datapath
wr_dbi_en_o	out	1	MR5 write DBI enable to datapath
ca_parity_lat_o	out	2	MR5 CA parity latency/mode to formatter
wrlvl_en_o	out	1	MR1 write-leveling enable to wrlvl_ifc
lpddr4_odt_o	out	TBD	LPDDR4 ODT/DQ-ODT programming to PHY/odt_ctrl

7.4 Microarchitecture internals

7.4.1 DDR4 MR semantics

The DDR4 mode-register set expands scoria's four registers to seven. The table below is semantics-binding; bit positions per JESD79-4 are confirmed at Q1 cold-storage read.

Table 2.7: DDR4 MR0-MR6 field semantics

Register	Fields (semantics-binding)
MR0	Burst length (fixed 8 or on-the-fly 4/8), read burst type (sequential/interleaved), CAS latency select, DLL reset bit, write recovery
MR1	DLL enable, additive latency (AL), RTT_NOM, write-leveling enable, TDQS enable, output driver impedance
MR2	CAS write latency (CWL), RTT_WR, write CRC mode bits (inert this edition per HAS Ch 3.1), LP ASR
MR3	MPR operation and page select, FGR refresh factor (1x/2x/4x), gear-down mode, MPR read format
MR4	Temperature status, preamble, CAL (command address latency)
MR5	Read DBI enable, write DBI enable, RTT_PARK, data-mask enable, CA parity latency/mode (A[2:0]), parity persistent-error (A9), parity error status (A4)
MR6	VrefDQ training range and value, tCCD_L select

Bit map per JESD79-4 MRn (confirmed at Q1 cold-storage read); the maps above are semantics-binding.

7.4.2 LPDDR4 MR set

LPDDR4 doesn't use the same MRS command as DDR4. Its MRs are written by MRW over the 6-bit CA bus and read back by MRR. The page owns the LPDDR4 write image alongside DDR4's. LPDDR4's MRs cover ODT and DQ-ODT programming, drive strength, CA training patterns, refresh-related bits, and the vendor-specific area. Exact LPDDR4 MR bit maps per JESD209-4 (Q1). The coupling is looser than DDR4's because termination is MR-programmed rather than ODT-pin driven, but the register-block structure is the same: one image per rank, one write port, one readback port.

7.4.3 MR coupling table

Table 2.8: Mode-register coupling

MR / Field	Consumer	What it controls
MR3 FGR select	`refresh_ctrl`	Refresh interval scaling: 1x, 2x or 4x
MR3 MPR page	`rdlvl_ifc`	MPR read-leveling page and format
MR2 RTT_WR	`odt_ctrl`	Write termination for DDR4
MR5 RTT_PARK	`odt_ctrl`	Idle termination for DDR4
MR5 RD/WR DBI	datapath (`dfi_rd_aligner`, `dfi_wr_serializer`)	Data Bus Inversion enable
MR5 CA parity	`dfi_cmd_formatter`	Parity latency and mode
MR1 write leveling	`wrlvl_ifc`	Write-leveling entry/exit
MR1 RTT_NOM	`odt_ctrl`	Nominal termination for DDR4
MR2 CWL	datapath / scheduler	CAS write latency (value flows to write-data timing)
MR6 VrefDQ	`rdlvl_ifc` / firmware	VrefDQ training range and value
LPDDR4 ODT/DQ-ODT	PHY / `odt_ctrl`	LPDDR4 MR-programmed termination

7.5 FSM policy

There is no FSM. The block is registers, a write port, a readback mux, and field fanout to the consumers above. Programming order is the init_sequencer's property (HAS Ch 3.2). Runtime MRW/MRR requests from firmware are single-cycle register updates; the formatter turns `mr_sel_o` and `mr_data_o` into the actual bus command.

7.6 Timing

MR writes and reads complete in one controller clock. The formatter and sequencer enforce the JEDEC command intervals (`tMRD`, `tMOD`) around MRS and MRW commands. Those values are runtime CSRs, initialised from the JESD79-4/JESD209-4 speed bin at CSR-derivation time (HAS Ch 5; numeric constants are HAS open question Q1).

7.7 Notes

- Keep the DDR4 and LPDDR4 image arrays in separate address spaces selected by `memtype_i`. Don't try to map them onto one flat array — the field widths and meanings differ enough that a unified map invites mistakes.
- The MR3 FGR factor and MPR page are the two fields most likely to be rewritten at runtime; expose them as named CSR fields even if the rest of the MR image is write-only at init.
- LPDDR4's MR-programmed termination means the `odt_ctrl` block is DDR4-scoped for this edition; the LPDDR4 ODT values still live here and fan out to the PHY configuration path.

8 Address Mapper (andesite_addr_mapper)

Module: andesite_addr_mapper.sv **Location:** rtl/fub/ (planned) **Category:** FUB **Parent:** scheduler_plane / cmd_arbiter **Status:** carried from scoria and landed — the tables below name the landed port lists

---

8.1 Purpose

`addr_mapper` turns the system address into the rank, bank-group, bank, row and column fields the scheduler and formatter need. The andesite change is one bounded addition: bank group (`BG0/BG1`) sits between chip select and bank. scoria's decode was rank, bank, row, column; andesite inserts bank group above bank for DDR4. LPDDR4 has no bank groups, so the group field is constant — degenerate, not special-cased.

8.2 Parameters

Table 2.9: Address mapper parameters

Parameter	Type	Range	Default	Meaning	Source
AXI_ADDR_WIDTH	int	32..48	32	host/ system address width	design point
NUM_RANKS	int	1, 2, 4	1	ranks per channel	design point
NUM_BANKS	int	4, 8	8	banks per rank	design point
BG_WIDTH	int	0..2	2	bank-group bits (0 for LPDDR4)	design point
ROW_WIDTH	int	14..18	14	row bits	design point
COL_WIDTH	int	10..12	10	column bits (without burst offset)	design point
BYTE_OFFSET_WIDTH	int	3..5	3	log2 of the beat byte size (8 B -> 3)	design point

8.3 Interface

Table 2.10: Address mapper ports

Signal	Direction	Width	Description
—	—	—	(no clock port: the mapper is pure combinational, carried from scoria)
—	—	—	(no reset port for the same reason)
`axi_addr_i`	in	`AXI_ADDR_WIDTH`	incoming system address
`bank_lsb_i`	in	5	bank field LSB in the word address (field-boundary knob)
`rank_o`	out	`$clog2(NUM_RANKS)`	decoded rank / chip select
`bg_o`	out	`BG_WIDTH`	decoded bank group (constant for LPDDR4)
`bank_o`	out	`$clog2(NUM_BANKS)`	decoded bank
`row_o`	out	`ROW_WIDTH`	decoded row
`col_o`	out	`COL_WIDTH`	decoded column (burst offset stripped)
`hash_en_i`	in	1	bank XOR-hash enable (carried from pumice; `HASH_EN` was never a parameter)
`hash_seed_i`	in	8	hash seed

8.4 Microarchitecture internals

8.4.1 Decode structure

The decode is parameterized and runtime-programmable via the ADDR_MAP-style CSR. Field boundaries reset to the design-point geometry and can be moved by firmware for characterization.

```
sys_addr -> { cs[CS-1:0],
              bg[BG-1:0]       (DDR4: BG0/BG1; LPDDR4: constant 0),
              bank[BANK-1:0]   (DDR4: 2 bits; LPDDR4: 3 bits),
              row[ROW-1:0],
              col[COL-1:0] }

Field boundaries are runtime CSRs (ADDR_MAP-style), reset to the
design point's
geometry; hash-en/hash-seed carried from pumice if enabled.
```

: Figure 2.2: Address decode structure (citation anchor)

memtype_i is deliberately absent: the decode is geometry-parameterized, not memtype-branched (the LPDDR4 degeneration is bg_o tied to zero by the consumer, per the note below the table). The ADDR_MAP-style field boundary is the bank_lsb_i knob, runtime-set, exactly as scoria's mapper. The bg_o delta is the andesite change; every other port is scoria's.

8.4.2 Scheduler-visible outputs

The mapper computes the group identity the arbiter gates on. For each candidate command, the scheduler sees {rank, bg, bank} and uses it for the same-group / cross-group checks described in ch02_blocks/05_scheduler.md. LPDDR4 has no bank groups, so bg_o is constant zero and the same-group check collapses to rank+bank — graceful degeneration, not a special case. The long/short timing pairs (tCCD_L/tCCD_S, tRRD_L/tRRD_S) still apply structurally; LPDDR4 simply makes L equal S.

8.4.3 Hashing

Address hashing is inherited from pumice and carried in scoria. When hash_en_i is set, a subset of the row/column bits is XOR-scrambled with hash_seed_i before the final field extraction. This spreads row-buffer contention across banks. The hash function is unchanged from the landed pumice implementation; andesite only widens the address path to include bank group.

8.5 FSM policy

There is no FSM. The block is pure combinational decode plus optional registered outputs to the scheduler. The fixed-form caution applies: this module inherits scoria's tested decode and extends it only in the named dimension — bank group between chip select and bank. No new hashing modes, no new address rearrangements.

8.6 Timing

Decode is combinational. Registered outputs add one controller-clock delay if needed to meet scheduler timing. Geometry is build-time per HAS Ch 2.4/5; field-boundary CSRs are runtime.

8.7 Notes

- The `bg_o` width is 2 for DDR4 and 0 for LPDDR4 at the parameter level. If zero-width ports are awkward in your RTL style, tie the LPDDR4 path to constant zero inside the module and keep the port width at 2 — but don't make the scheduler special-case it.
- The design point fixes DDR4-1600 x8 with 4 bank groups × 4 banks and LPDDR4-1600 x16 with 8 ungrouped banks per channel. Reset the `ADDR_MAP` CSRs to those geometries so firmware doesn't have to.
- scoria's decode equations are the starting point; the kmap book's first address-map targets are the verification targets for andesite.

9 Scheduler and Command Arbiter (andesite_scheduler_plane + andesite_cmd_arbiter)

Module: andesite_scheduler_plane.sv, andesite_cmd_arbiter.sv **Location:** rtl/macro/ (scheduler), rtl/fub/ (arbiter) **Category:** scheduling / arbitration **Parent:** andesite_core **Status:** carried from scoria and landed — arbiter carries the L/S delta; the tables below name the landed port lists. The macro's init/mode-register instances are rewired to the P1 blocks (macro-integration pass, 2026-10-05): the P1 init_sequencer is self-timed and DDR4-only, and the P1 mode_register policy outputs are the macro's observability contract.

9.1 Purpose

The scheduler picks which ready command goes to the formatter each cycle. It inherits scoria's FR-FCFS arbiter and its maintenance request/grant channel. The andesite delta is L/S-aware issue: DDR4 bank groups add a same-group versus cross-group split to the command-spacing checks, and the global timers grow the `tCCD_L/S` and `tRRD_L/S` pairs. LPDDR4 has no bank groups, so the long and short checks collapse to one.

The scheduler never invents a new arbitration philosophy. It takes scoria's mechanism and teaches it about bank groups.

9.2 Parameters

Table 2.5.1: Scheduler parameters

Parameter	Type	Range	Default	Meaning
NUM_RANKS	int	1+	1	ranks per channel
NUM_BANKS	int	8..16	8	banks per rank
NUM_BG	int	1..4	4	bank groups; 1 is the LPDDR4 degeneration (the L/S delta parameter)
NUM_ENTRIES	int	8..32	8	read and write CAM depth
AGE_WIDTH	int	—	16	age-counter width

`NUM_BG = 1` is how LPDDR4 degenerates gracefully. There is no `if (memtype == LPDDR4)` branch in the issue check; the group index is constant and the L/S counters collapse by construction.

9.3 Interface

9.3.1 Host-side command ports

Signal	Direction	Width	Description
aclk	in	1	controller clock
aresetn	in	1	active-low reset
init_done_i	in	1	scheduler admits no host commands until high
rd_sch_valid_i	in	NUM_ENTRIES	read CAM per-entry valid (the CAMs live in the intake/AXI interface)
wr_sch_valid_i	in	NUM_ENTRIES	write CAM per-entry valid
rd_sch_bank_i	in	NUM_ENTRIES*BKW	read candidate bank per entry; rd_sch_row_i/rd_sch_col_i carry the rest of the payload
wr_sch_bank_i	in	NUM_ENTRIES*BKW	write candidate bank per entry; wr_sch_row_i/wr_sch_col_i alongside
rd_sch_older_i	in	NUM_ENTRIES^2	read CAM age-order matrix; the write side is wr_sch_older_i
rd_issue_valid_o	out	1	read command wins this cycle; rd_issue_slot_o names the

Signal	Direction	Width	Description
			winning entry
wr_commit_valid_o	out	1	write command wins this cycle; wr_commit_slot_o names the winning entry
cmd_valid_o	out	1	issued command strobe toward the formatter path; cmd_op_o, cmd_rank_o, cmd_bank_o, cmd_row_o, cmd_col_o, cmd_ap_o carry it, cmd_ready_i back-pressures
cmd_bg_o	out	$clog2(NUM_BG)	bank group of the issued command (the arbiter's registered-pick group, bank[BKW-1 -: BGW]); rides the widened command word {ap,col,row,bg,bank,rank,op} to the DFI formatter

9.3.2 Bank-group timing ports

Signal	Direction	Width	Description
tccd_ok_i	in	1	tCCD base spacing ok (cross-group; the LPDDR4 degenerate check)
tccd_l_ok_i	in	1	tCCD_L same-

Signal	Direction	Width	Description
			group spacing ok — the andesite L/S delta input
trrd_ok_i	in	1	tRRD_S-class ACT-to-ACT ok, per rank
trrd_l_ok_i	in	1	tRRD_L same-group ACT-to-ACT ok — L/S delta input
tfaw_ok_i	in	1	tFAW four-activate window ok, per rank
twtr_ok_i	in	1	write-to-read turnaround ok; trtw_ok_i is the read-to-write partner
bank_act_ready_i	in	NUM_RANKS x NUM_BANKS	per-bank timer readiness (activate); bank_rdwr_ready_i, bank_pre_ready_i and the _la_i lookahead variants ride alongside

The L/S counters live in the global timers; the arbiter consumes the ok flags. The bank group of the last-issued command is tracked inside the arbiter (r_last_col_bg, r_last_act_bg), not on a port — there is no last_cmd_group signal.

9.3.3 Maintenance request/grant ports

Signal	Direction	Width	Description
refresh_req_i	in	1	refresh wants the

Signal	Direction	Width	Description
			bus (from `refresh_ctrl`)
`refresh_grant_o`	out	1	bus granted to refresh
`refresh_drain_i`	in	1	refresh drain in progress; CAMs hold
`refresh_kind_i`	in	1	0 = all-bank `REF`, 1 = per-bank (LPDDR4)
`refresh_bank_i`	in	`BKW`	per-bank target mirror (the rotor)
`zq_req_i`	in	1	ZQ calibration wants the bus (from `zq_ctrl`)
`zq_grant_o`	out	1	bus granted to ZQ

The carried RTL keeps scoria's two request/grant pairs — one for refresh, one for ZQ — rather than one unified `maint_req`/`maint_tag` channel; the unified tagged channel stays the conceptual model, and the formatter-facing tag is derived downstream. The ODT turnaround request lands with `odt_ctrl` (Ch 2.8). The rule is inherited from scoria: request and wait, never preempt.

Table 2.5.2: Scheduler interface

9.4 Macro init/policy interface (P1 rewiring, landed 2026-10-05)

The scheduler macro owns the P1 `init_sequencer` and `mode_register` instances. The scoria DDR3-era connections (`mc_clk`-style pins, the `dfi_init_start`/`dfi_init_complete` PHY handshake, and the CL/CWL/BL shadow outputs) retired with the rewiring — the P1 sequencer is self-timed (`tINIT*` CSRs; any non-DDR4 `memtype` latches `init_err`) and the P1 register never carried latency shadows.

Signal	Direction	Width	Description
`t_init_wait_i`	in	16	`tINIT1` — RESET# low time
`t_rp_wait_i`	in	8	`tINIT3` — RESET# release to CKE (zero-extended)
`t_cke_wait_i`	in	16	`tINIT4` — CKE to first MRS
`t_mrd_wait_i`	in	8	`tMRD` — MRS-to-MRS gap (zero-extended)
`t_mod_wait_i`	in	16	`tMOD` — last MRS to ZQCL
`t_dll_wait_i` / `t_zqinit_wait_i`	in	16	`tDLLK` / `tZQinit`
`mr0_i` … `mr6_i`	in	16 ×7	DDR4 MR images; the P1 FSM writes MR3, MR6, MR5, MR4, MR2, MR1, MR0
`dram_reset_n_o` / `cke_o`	out	1	DRAM RESET# / CKE pins, driven by the P1 init FSM
`init_done_o` / `init_err_o`	out	1	init complete / unsupported-memtype or watchdog error
`zq_cal_start_o`	out	1	status strobe marking the init

Signal	Direction	Width	Description
			FSM's ZQCL issue (the ZQCL itself rides the command stream)
gear_down_entry_o / ca_train_start_o / parity_enable_o	out	1	P1 init-FSM status outputs (parity latches on at MR5 per the init sequence)
rtt_nom_o / rtt_wr_o / rtt_park_o	out	3	ODT policy images (the odt_ctrl observability contract)
rd_dbi_en_o / wr_dbi_en_o	out	1	MR5 read/write DBI enables (the DFI datapath consumers)
mpr_page_o / fgr_factor_o / ca_parity_lat_o / lpddr4_odt_o	out	2 / 2 / 2 / 3	MR3 MPR page, FGR factor, CA-parity latency, LPDDR4 ODT image
wrlvl_en_o	out	1	MR1[7] — write-leveling mode authority (to the training plane)

Init commands (MRS / ZQCL) reach the DRAM through the ordinary command stream: the P1 cmd_req/cmd_ack handshake is honored by the arbiter's accept pulse (cmd_ack = accept qualified by OP_MRS/OP_ZQCL), and the arbiter forwards the init payload (cmd_addr low bits) on the stream's row field, exactly as the carried scoria forwarding did. The init MR index rides cmd_bank into the mode-register store's write address.

9.5 Microarchitecture internals

9.5.1 FR-FCFS admission with L/S awareness

scoria's FR-FCFS arbiter is unchanged in spirit. Each candidate command is scored by age and readiness, and the highest scorer that passes the timing checks is granted. The andesite change is inside the timing check.

```
issue_ok(cmd) = bank_timers.ok(cmd)
                AND global_timers.ok(cmd, group(cmd), group(last_cmd))
```

`group(cmd)` is the bank group of the candidate's bank. For a read or write command, the global-timer check now depends on whether the candidate targets the same bank group as the last-issued command:

- Same group: `tCCD_L` and `tRRD_L` apply.
- Different group: `tCCD_S` and `tRRD_S` apply.

The check also covers the outstanding same-group spacing obligations a burst of commands to one group leaves behind; the last-issued group registers (`r_last_col_bg`, `r_last_act_bg`) and the L/S ok flags carry that state into the admission check.

9.5.2 Global timers

The global timer block inherits scoria's counters and adds four new ones. All are runtime CSRs.

Counter	JEDEC symbol	Applies
`tCCD_L`	`tCCD_L`	same bank group
`tCCD_S`	`tCCD_S`	different bank groups
`tRRD_L`	`tRRD_L`	same bank group, activate-to-activate
`tRRD_S`	`tRRD_S`	different bank groups, activate-to-activate

These join the inherited `tCCD`, `tRAS`, `tRC`, `tRP`, `tRCD`, `tWTR`, `tRTW`, `tWR`, and the read/write turnaround set. The values are runtime CSRs, initialised from the JESD79-4/JESD209-4 speed bin at CSR-derivation time (HAS Ch 5; numeric constants are HAS open question Q1).

9.5.3 Maintenance request/grant semantics

The carried RTL keeps scoria's channel shape: two request/grant pairs into the arbiter, one for refresh (`refresh_req_i`/`refresh_grant_o`) and one for ZQ (`zq_req_i`/`zq_grant_o`). The original spec sketch on this page was one unified `maint_req` with a `maint_tag` source identifier;

the carry ruled that the inherited pairs are the truth this tier, and the tag survives only as the conceptual model the formatter-side maintenance encoding derives from.

Source	Arbiter channel	Action when granted
`refresh_ctrl`	`refresh_req_i`/ `refresh_grant_o`	issue `REF` or LPDDR4 per-bank refresh
`zq_ctrl`	`zq_req_i`/`zq_grant_o`	issue `ZQCS`, `ZQCL`, or LPDDR4 MPC ZQ calibration
`andesite_training_plane`	`trn_cmd_req_i`/ `trn_cmd_grant_o`	issue training `MRS` or `MPC`; request-and-wait, all-banks-idle two-step shape like refresh
`odt_ctrl`	lands with Ch 2.8	issue the ODT turnaround command or NOP-with-ODT

The rule is inherited from scoria: request and wait, never preempt. A maintenance source raises its `req` and holds it until `grant` arrives. It cannot yank the bus away from an in-flight host command.

9.5.4 LPDDR4 degeneration

LPDDR4 has no bank groups, so `NUM_BANK_GROUPS = 1` and `group(cmd)` is always zero. The same-group and cross-group checks collapse into a single check per pair because there is only one group to compare against. The L/S counters still exist, but they are loaded with the same value. This must fall out of the parameterization, not out of an `if (memtype == LPDDR4)` branch.

This is worth stating plainly: LPDDR4 is not a special case in the arbiter. It is the `NUM_BANK_GROUPS = 1` corner of the same code.

9.6 FSM policy

The arbiter is priority logic plus CAMs, not an FSM. That is inherited from scoria. The scheduler's grant sequencing — one command per cycle, maintenance highest, FR-FCFS among host commands — is the same mechanism with the L/S inputs added.

The global timers are counters, not a state machine. Each timer decrements every cycle and asserts its ok flag at zero.

9.7 Timing

One grant per cycle, as inherited. The new counters add combinational delay to the admission check but no new pipeline stages. The critical path is:

1. Read CAM and write CAM produce candidates.
2. Bank timers check per-bank readiness.
3. Global timers check L/S spacing; the last-issued group registers select the long or short window.
4. FR-FCFS priority selects the winner.
5. Grant is registered and the command is issued.

The counter values are runtime CSRs, initialised from the JESD79-4/JESD209-4 speed bin at CSR-derivation time (HAS Ch 5; numeric constants are HAS open question Q1).

9.8 Notes

- **Why pairs, not a unified tagged channel:** the carried RTL is scoria's two request/grant pairs, kept byte-behavioral. The scheduler doesn't need to know what kind of maintenance it is granting; it only needs to know that maintenance beats host traffic and that only one source wins per cycle. A unified `maint_req`/`maint_tag` channel stays on the books as the conceptual model, not the landed interface.
- **L/S vs the old `tCCD`:** The inherited `tCCD` is kept as a fallback for modes where bank-group information is not available, but the primary check is the L/S pair. DV must prove that `tCCD_L` and `tCCD_S` subsume the old `tCCD` correctly for DDR4 and collapse to it for LPDDR4.
- **Maintenance starvation:** Because maintenance never preempts, a pathological stream of host commands could delay refresh or ZQ. The refresh controller's credit window and the ZQ controller's overdue counter handle this at the source; the scheduler's only obligation is fair arbitration once the request is raised.

9.9 cmd_history_checker's DDR4 growth (landed 2026-10-04)

The checker's spacing-parameter set now carries the long/short pairs the bank-group scheduler defines: `T_CCD_L`/`T_CCD_S` (CAS-to-CAS within / across a bank group) and `T_RRD_L`/`T_RRD_S` (ACT-to-ACT), all runtime parameters inert at 0, plus a `cmd_bg_i` bank-group command input. scoria's nine checks are byte-identical; checks (10)-(13) are additive. The ported suite proves each new check fires with its own fatal text (`fires_tccd_l/s`, `fires_trrd_l/s`) and that legal streams at both minima stay silent (`legal_ls_pairs_at_the_minimum`).

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 Refresh Controller (andesite_refresh_ctrl)

Module: andesite_refresh_ctrl.sv **Location:** rtl/fub/ **Category:** maintenance / refresh
Parent: andesite_core **Status:** carried from scoria and landed, FGR delta included — the ports table names the landed list; the fence below is the design citation (the generator diffs it)

10.1 Purpose

`refresh_ctrl` keeps the DRAM rows alive. It inherits scoria's verified base, specified in `projects/components/mem-ctrl-ip/research-ip/scoria-ddr3-lpddr3/docs/scoria_has/ch03_architecture/04_refresh.md` and proven in `formal/scoria/refresh_ctrl`: the `tREFI` interval counter, all-bank refresh, JEDEC's plus-or-minus-eight postpone/pull-in credit window, and the TASK-001 policy modes (A: elastic refresh, B: temperature-compensated refresh, C: placement discipline). The andesite deltas are DDR4 fine-granularity refresh and LPDDR4 controller-directed per-bank refresh.

The block is MODIFIED, not rewritten. Every inherited property transfers with the mechanism.

10.2 Parameters

Table 2.6.1: Refresh controller parameters

Parameter	Type	Range	Default	Meaning
NUM_BANKS	int	8..16	8	banks per rank
BA_W	int	—	$clog2(NUM_BANKS)	bank-address width (drives refresh_bank_o)

10.3 Interface

Table 2.6.2: Refresh controller ports

Signal	Direction	Width	Description
mc_clk	in	1	controller clock
mc_rst_n	in	1	active-low reset
enable_i	in	1	refresh engine enable
t_refi_i	in	16	tREFI reload value, runtime CSR; the FGR delta divides it by the factor (below)
refi_reload_i	in	1	reload the countdown immediately (DV/bring-up pulse; tie 0 in production)
trefi_pb_i	in	16	per-bank tREFI for the LPDDR4 REFpb path
refresh_burst_i	in	4	refreshes owed per request cycle (the JEDEC burst)
refpb_mode_i	in	1	0 = all-bank REF, 1 = controller-directed per-bank refresh
demand_i	in	1	demand indication from the scheduler side
elastic_en_i	in	1	Mode A elastic refresh enable; pullin_idle_stre

Signal	Direction	Width	Description
			ak_i (8) and postpone_demand_streak_i (7) ride as its thresholds
tcr_en_i	in	1	Mode B temperature-compensated refresh enable; trefi_derate_i (2) is its derate select, reload-only as inherited
fgr_factor_i	in	2	DDR4 FGR factor from the MR3 image: 0 = 1x, 1 = 2x, 2 = 4x; an illegal encoding clamps to the 1x row (the kmap FGR select table — deliberately not the Mode B 3→2 clamp)
t_rfc_1x_i	in	16	tRFC at 1x density, runtime CSR; t_rfc_2x_i and t_rfc_4x_i carry the 2x/4x densities — the factor selects among them on refresh_trfc_o
postpone_limit_i	in	4	credit-window postpone bound (JEDEC +8); pullin_limit_i is

Signal	Direction	Width	Description
			the pull-in bound
refresh_req_o	out	1	request to the scheduler; held until refresh_grant_i — request-and-wait, never preempt
refresh_grant_i	in	1	scheduler grant
grant_was_pb_i	in	1	the granted command was per-bank
refresh_drain_active_o	out	1	drains in-flight commands before the refresh issues
refresh_kind_o	out	1	0 = all-bank, 1 = per-bank
refresh_bank_o	out	BA_W	per-bank refresh target (the rotor output)
pending_refreshes_o	out	4	credit-window occupancy (postpones owed)
obs_refi_cnt_o	out	16	interval-countdown observability; obs_drain_remaining_o, obs_bank_rotor_o, obs_grants_total_o, obs_pullin_credit_o, obs_postpone_events_o,

Signal	Direction	Width	Description
refresh_trfc_o	out	16	obs_pullin_events_o complete the telemetry set tRFC_active = tRFC(fgr), the factor-selected recovery window; consumed by the recovery owner (the arbiter) when the macro rewiring lands

The FGR delta is reload-only: a factor change mid-interval takes effect on the next reload, never the running counter — the same posture Mode B carries. The factor and the per-density tRFC CSRs are ports, not parameters.

10.4 Microarchitecture internals

10.4.1 Inherited mechanism

The base mechanism is scoria's `refresh_ctrl`, proven in `formal/scoria/refresh_ctrl`. It carries:

- A `tREFI` interval counter.
- The JEDEC ±8 postpone/pull-in credit window.
- Mode A elastic streaks: `pullin_idle_streak` reset to 16, `postpone_demand_streak` reset to 1.
- Mode B TCR derate applied to the counter reload only.
- `REF_STATS` telemetry.
- The sixteen-cycle idle confirmation, because CAM occupancy blinks off between bursts and those micro-gaps must not be treated as idle.

The credit ceiling is ±8 in every mode. Mode A and Mode B touch scheduling policy, not the credit arithmetic. ### DDR4 delta: fine-granularity refresh

DDR4's MR3 selects 1x, 2x, or 4x refresh granularity. The controller scales its interval arithmetic by the FGR factor.

```
fgr_factor = {1x:1, 2x:2, 4x:4}   (MR3 image, runtime CSR)
tREFI_effective = tREFI / fgr_factor   (counter reload value)
tRFC_active = tRFC(fgr)                 (CSR per density)
credits window scales with the density bookkeeping — the ±8 ceiling is
unchanged.
```

`fgr_factor_i` is the runtime copy of the MR3 field that `init_sequencer` programmed (0 = 1x, 1 = 2x, 2 = 4x; illegal clamps to the 1x row per the kmap FGR table — deliberately not the Mode B 3→2 clamp). The `tREFI` counter reloads with `tREFI_effective`. The per-refresh recovery window is `tRFC_active`, selected from the `t_rfc_1x_i`, `t_rfc_2x_i`, or `t_rfc_4x_i` CSR by the factor and exposed on `refresh_trfc_o` for the recovery owner; the scaling is reload-only, so a factor change mid-interval takes effect on the next reload.

The credit-window bookkeeping scales with the density accounting, but the ±8 ceiling is unchanged. Postponing 8 refreshes at 4x granularity is not the same as postponing 8 refreshes at 1x; the bookkeeping must track effective refreshes, not raw commands.

10.4.2 LPDDR4 delta: controller-directed per-bank refresh

LPDDR4 changes who names the bank. For LPDDR2/3, `REFpb` carried no bank address and scoria's rotor mirrored the device's fixed counter. JESD209-4 lets the controller put the bank on the CA bus, so the rotor is replaced by explicit bank scheduling.

This edition implements a round-robin default. The policy plane that scoria's TASK-001 built — CSR-select, reset-off, telemetry-instrumented — is exactly where occupancy-aware schemes from andesite TASK-001's DARP survey would hang. Round-robin is the commodity choice; advanced schemes are left as future policy hooks.

The scheduling policy hook picks the next bank. On the carried block the hook is internal: a round-robin rotor advances the per-bank pointer and drives `refresh_bank_o` (observable on `obs_bank_rotor_o`), with no bank-busy input — the busy-masked, occupancy-aware variants are the future policy this page leaves hooked, not a port the inherited engine has. Future policies can replace the hook without touching the refresh engine.

10.5 FSM policy

The refresh engine's FSM is inherited in shape: request/grant, issue, then track the busy interval. The FGR factor and per-bank selection are inputs, not new states.

Mode selects are data, not control. Whether the current refresh is 1x or 4x, all-bank or per-bank, round-robin or occupancy-aware — these choices change the values loaded into counters and the bank address driven to the scheduler. They do not change the state machine's skeleton. That's why the FSM stays inherited.

10.6 Timing

The `tREFI` counter reloads with `tREFI_effective`, a runtime CSR derived from the JESD79-4/JESD209-4 speed bin at CSR-derivation time (HAS Ch 5; numeric constants are HAS open question Q1). `tRFC_active` is selected from the FGR-specific CSR.

The Mode B TCR derate applies to the reload value only: a mid-interval CSR change takes effect on the next reload, not on the running counter. That's the same reload-only rule inherited from scoria.

10.7 Notes

- **Formal obligation:** Retention must be re-derived per FGR factor and for the per-bank arithmetic. Carrying the 1x all-bank proof forward unchanged would prove the wrong thing while appearing green. The obligation is recorded here and in HAS Ch 6 item 5.
- **Refresh is an ordinary bank occupant:** Bank timers own the banks during refresh. A refresh request is not a special bypass; it waits its turn in the scheduler like any other command.
- **Why round-robin only:** Occupancy-aware per-bank refresh is implementable now that the controller names the bank, but it is a policy change, not a mechanism change. The policy hook exists; the survey's schemes stay in andesite TASK-001.
- **Telemetry:** `REF_STATS` carries over. `ref_stats_fgr_factor` and `ref_stats_perbank_bank` join it so a refresh sweep is measurable in-system rather than inferred.

11 ZQ Calibration Controller (andesite_zq_ctrl)

Module: andesite_zq_ctrl.sv + andesite_zq_mpc_lpddr4.sv (both landed) **Location:** rtl/fub/ **Category:** maintenance / calibration **Parent:** andesite_core **Status:** DDR4 core carried from scoria and landed; the LPDDR4 MPC submodule is landed per the FSM below; ports tables reconciled to the landed port lists

---

11.1 Purpose

`zq_ctrl` issues periodic ZQ calibration. For DDR4, the block is scoria's module carried unchanged: interval CSR, request-and-wait grant, `tZQCS` enforced by holding the next interval, issued-calibration counter, and CSR-selectable placement policy. For LPDDR4, calibration moves onto the MPC command, so a new submodule sequences it through the formatter's LPDDR4 CA path.

The DDR4 side is INHERITED. The LPDDR4 side is NEW. Both share one maintenance request port to the scheduler with a source tag.

11.2 Parameters

Table 2.7.1: ZQ controller parameters

Parameter	Type	Range	Default	Meaning
—	—	—	—	(no parameters; the interval and overdue widths are fixed on the ports — 32-bit interval, 13-bit overdue)

11.3 Interface

11.3.1 Common ports (the landed DDR4 core)

Signal	Direction	Width	Description
mc_clk	in	1	controller clock
mc_rst_n	in	1	active-low reset
enable_i	in	1	calibration engine enable (ZQ_CFG.zq_enable)
memtype_i	in	3	memtype_e: selects the calibration path — MEMTYPE_LPDDR4 hands expiries to the MPC submodule; anything else runs the inherited DDR4 core
t_zqcs_interval_i	in	32	calibration interval, runtime CSR; 0 = off (shared by both paths)
t_zqcs_i	in	16	tZQCS recovery, runtime CSR (DDR4 path)
placement_i	in	2	placement policy select (Mode C, DDR4 path)
overdue_max_i	in	13	defer-under-demand threshold (DDR4 path)
demand_i	in	1	demand

Signal	Direction	Width	Description
			indication from the scheduler side
zq_req_o	out	1	request to the scheduler; held until zq_grant_i — request-and-wait, never preempt (muxed between the core and the submodule)
zq_grant_i	in	1	scheduler grant
obs_busy_o	out	1	a calibration is in flight (post-grant window, either path)
obs_zqcs_total_o	out	16	issued-calibration counter (shared: counts either path's completion)
obs_interval_cnt_o	out	32	interval countdown observability (shared by both paths)
obs_overdue_o	out	1	overdue flag (DDR4 placement-policy telemetry; the LPDDR4 path does not defer, so it stays low there)

The carried DDR4 core exposes no DDR4-only ports beyond this set: ZQCS is its only calibration flavour, and the init-time long calibration is sequenced by the scheduler macro's t_zqinit_wait_i window, not by this block. The original spec sketch's init_zqcl_req/init_zqcl_done handshake and the unified maint_req/maint_tag channel

are not how the carried RTL works — the core requests calibration on `zq_req_o` and waits on `zq_grant_i`.

11.3.2 LPDDR4-only ports (the landed MPC submodule)

The submodule is `andesite_zq_mpc_lpddr4` (its own FUB file); `zq_ctrl` instantiates it and memtype-muxes the scheduler pair and the busy observable — a combinational passthrough of the two registered sources, so each path keeps single-register latency.

Table 2.7.2: ZQ controller ports

Signal	Direction	Width	Description
`t_zq_i`	in	16	`tZQ` calibration latency, runtime CSR (JESD209-4 speed-bin derived); waited out in `MPC_WAIT`
`mpc_opcode_i`	in	6	MPC opcode **image** from the CSR. The encodings are TBC(JESD209-4): the submodule drives the image and never decodes it — no MPC opcode encodings are invented anywhere in andesite
`mpc_issuing_o`	out	1	high in `MPC_ISSUE`; the formatter samples the image with the grant cycle
`mpc_op_o`	out	6	the opcode image presented while issuing (follows

Signal	Direction	Width	Description
			`mpc_opcode_i` combinationally)

11.4 Microarchitecture internals

11.4.1 DDR4 path: inherited from scoria

The DDR4 core is scoria's `zq_ctrl`, unchanged. It implements:

- Interval CSR: countdown to the next calibration event.
- Request-and-wait grant: it asks the scheduler and waits; it never preempts.
- `tZQCS` enforcement: the next interval is held until the recovery time expires.
- Issued-calibration counter: tracks how many calibrations have gone out.
- CSR-selectable placement policy: scoria TASK-001 Mode C, baseline versus defer-under-demand with `overdue_max`.

For the internal details, see scoria's HAS Ch 3.2 and the landed RTL in `projects/components/mem-ctrl-ip/research-ip/scoria-ddr3-lpddr3/rtl/zq/`. This MAS page does not re-specify what already exists; it cites it.

11.4.2 LPDDR4 path: new MPC submodule

LPDDR4 carries calibration on the MPC command, not dedicated `ZQCS`/`ZQCL` opcodes. The new submodule issues the ZQCal start and ZQCal latch opcodes through the formatter's LPDDR4 CA submodule.

```
MPC_IDLE
    -> MPC_ISSUE   (assert cmd_req, wait for scheduler grant, drive
MPC ZQCal start)
    -> MPC_WAIT    (wait tZQ latched from CSR)
    -> MPC_DONE    (drive calibration complete, reload interval
counter)
    -> MPC_IDLE
```

`tZQ` here is the LPDDR4 ZQ calibration latency, the `t_zq_i` runtime CSR loaded from JESD209-4 at CSR-derivation time. The submodule rides the core's shared interval counter and telemetry: `obs_zqcs_total_o` counts either path's completion, `obs_interval_cnt_o` is the live countdown, and `MPC_DONE` is what reloads the shared interval.

The MPC submodule depends on the LPDDR4 CA path in `dfi_cmd_formatter` (Ch 2.1). It does not build its own CA encoder; it drives the MPC opcode and lets the formatter place it on the 6-bit CA bus.

11.4.3 Scheduler interface

`zq_ctrl` presents its calibration request on the carried pair, `zq_req_o`/`zq_grant_i`, alongside refresh's own pair. The scheduler does not special-case ZQ versus refresh; each source waits its turn. See `05_scheduler.md` for the maintenance request/grant table.

11.5 FSM policy

The inherited DDR4 core keeps whatever FSM it already has. This page does not invent a new one for it.

The LPDDR4 MPC submodule is a new small FSM with the four states listed above: `MPC_IDLE`, `MPC_ISSUE`, `MPC_WAIT`, `MPC_DONE`. It is not nested under the DDR4 core; it is a sibling path selected by `MEMTYPE`.

11.6 Timing

For DDR4, `tZQCS` and `tZQCL` are runtime CSRs enforced by the inherited core. For LPDDR4, `tZQ` is a runtime CSR, initialised from the JESD209-4 speed bin at CSR-derivation time (HAS Ch 5; numeric constants are HAS open question Q1), and the MPC submodule waits it out in `MPC_WAIT`.

The interval counter reloads when calibration completes. A mid-interval CSR change takes effect on the next reload, following the same reload-only rule used elsewhere.

11.7 Notes

- **Init-time ZQCL:** the carried core has no init handshake. The init-time long calibration is sequenced by the scheduler macro's `t_zqinit_wait_i` window (and the init MRS/ZQ chain in the macro); this block owns periodic `ZQCS` from `t_zqcs_interval_i`. When the MPC submodule lands it takes over the LPDDR4 side of that chain.
- **Telemetry sharing:** `obs_zqcs_total_o`, `obs_interval_cnt_o`, and `obs_overdue_o` are the core's counters. The MPC submodule will share them when it lands; the two paths never operate at once because the memtype selects one build, so there is no arbitration conflict.
- **Placement policy:** the `overdue_max_i`/`placement_i` policy is inherited for DDR4 and carried conceptually for LPDDR4, but this edition only exercises the default path for LPDDR4 MPC calibration. Future policy hooks use the same CSR surface.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

12 ODT Controller (andesite_odt_ctrl)

Module: andesite_odt_ctrl (landed, NEW) **Location:** projects/components/mem-ctrl-ip/research-ip/andesite-ddr4-lpddr4/rtl/fub/ **Category:** termination policy / rank control **Parent:** andesite_top / scheduler boundary **Status:** landed, NEW — the ports table names the landed list; the fence below is the design citation

12.1 Purpose

`odt_ctrl` turns DDR4's termination schedule into a per-rank ODT pin policy. The DRAM exposes three termination values — RTT_NOM, RTT_WR, and RTT_PARK — and a set of latencies that say when each one takes effect. This block watches the scheduler's grant stream, decides which rank should terminate how, and drives the ODT pins with the right delay. It is a tap, not an arbiter; it never stalls a command to make a termination point.

12.2 Parameters

Table 2.8.1: `odt_ctrl` parameters

Parameter	Type	Range	Default	Meaning	PRD
NUM_RANKS	int	1..4	1	ranks sharing the DQ bus	A1
RTT_NOM_IMG	CSR field	vendor-defined	TBD	MR1 RTT_NOM image for this board	A2
RTT_WR_IMG	CSR field	vendor-defined	TBD	MR2 RTT_WR image for this board	A3
RTT_PARK_IMG	CSR field	vendor-defined	TBD	MR5 RTT_PARK image for this board	A4
ODTLon_CSR	CSR	speed-bin range	TBD	assertion latency from command to ODT active	A5
ODTLoff_CSR	CSR	speed-bin range	TBD	de-assertion latency from command to ODT inactive	A6
ODT_TURN_CSR	CSR	speed-bin range	TBD	write-to-read ODT turnaround value	A7

Parameter	Type	Range	Default	Meaning	PRD
tADC_CSR	CSR	speed-bin range	TBD	command-to-ODT-change delay for transitions	A8

The numeric values behind these CSRs come from the selected JESD79-4 speed bin at CSR-derivation time (HAS Ch 5; numeric constants are HAS open question Q1).

12.3 Interface

Table 2.8.2: `odt_ctrl` ports

Signal	Direction	Width	Description
`mc_clk`	in	1	controller clock
`mc_rst_n`	in	1	active-low reset
`init_done_i`	in	1	ownership transfers from `init_sequencer` at this edge; `init_odt_i` is sampled as the starting pin value
`init_odt_i`	in	`NUM_RANKS`	ODT value held by `init_sequencer` until handoff
`grant_valid_i`	in	1	the grant tap is valid this cycle
`grant_op_i`	in	5	granted command (`dram_op_e`: RD/RDA/WR/WRA decode; anything else holds state)
`grant_rank_i`	in	`RKW`	rank selected by the grant
`odt_pin_o`	out	`NUM_RANKS`	per-rank ODT pin toward the PHY/DRAM
`odtlon_i`	in	8	assertion latency, command to ODT active
`odtloff_i`	in	8	de-assertion latency, burst to ODT inactive
`odt_turn_i`	in	8	write-to-read ODT

Signal	Direction	Width	Description
			turnaround
`tadc_i`	in	8	command-to-ODT-change delay (also the handoff settle)
`rtt_nom_img_i`	in	3	MR1 RTT_NOM image — a value contract (the RTT values are init-side); `rtt_wr_img_i`/`rtt_park_img_i` (MR2/MR5) ride alongside
`hist_state_o`	out	2 per rank	per-rank ODT policy state (packed)
`trans_count_o`	out	8 per rank	per-rank transition counters (packed)

12.4 Microarchitecture internals

12.4.1 Policy state table

The termination policy is per-rank. Each rank is in one of four policy states, selected from the grant stream (the landed RTL names:

```
state        | RTT applied   | entered when
PARK         | RTT_PARK      | no rank selected (park policy)
RD_NOM       | RTT_NOM       | a read granted to another rank (this
rank terminates)
RD_SELF      | ODT off       | a read granted to this rank (its own
ODT stays off)
WR           | RTT_WR        | a write granted to this rank
(per-rank; values are the MR1/MR2/MR5 images, CSR-programmed)
```

`RD_NOM` means a read granted to another rank makes this rank present RTT_NOM. `RD_SELF` means a read granted to this rank keeps its own ODT off. `WR` means a write granted to a rank makes that rank present RTT_WR. A single-rank design point exercises PARK and WR (`RD_SELF` is the read case at one rank); the multi-rank coupling is already in the structure and is not special-cased away.

12.4.2 Command-stream tap

`odt_ctrl` consumes the scheduler's grant stream: which rank won, and whether it won with a read or a write. That is all the block needs. It does not arbitrate, it does not back-pressure, and it does not modify the command stream. The HAS requires this explicitly (HAS Ch 3.5, requirement 3): ODT transitions never stall commands.

12.4.3 Pin timing

ODT assertion and de-assertion are delayed from the command by the ODTL family. ODTLon governs how many clocks after a write command the ODT pin asserts; ODTLoff governs how many clocks after the burst the pin releases. The write-to-read ODT turnaround is the gap when switching from a write termination picture to a read termination picture, and tADC is the command-to- ODT-change interval used for rank-to-rank transitions. All four are runtime CSRs, named by their JEDEC symbols, initialised from the JESD79-4 speed bin at CSR-derivation time (HAS Ch 5; numeric constants are HAS open question Q1).

A small counter per rank loads the relevant latency when the grant decodes to a state change and counts down to the pin edge. The counter can be shared between assertion and release because the block never has overlapping state machines on the same rank in this policy.

12.4.4 The init seam

Until `init_done`, the ODT pin belongs to `init_sequencer`. The init sequence programs MR1/MR2/MR5 and holds ODT per the JEDEC initialization constraint; after the final mode-register

writes it waits at least tMOD before raising `init_done`. At that cycle ownership transfers to `odt_ctrl`: the block samples `init_odt` as its starting pin value and from then on drives the pin from its own policy state. The handoff is a named, bounded seam.

12.4.5 LPDDR4 delta

LPDDR4 has no ODT pin. DQ ODT is programmed in the LPDDR4 mode-register set at init and held static this edition — it lives in `mode_register`'s LPDDR4 maps and in the init sequence, not in `odt_ctrl`. The dynamic machinery above is DDR4-scoped; the block's structure keeps that scoping explicit rather than pretending a pin LPDDR4 doesn't have.

12.4.6 Telemetry

`odt_ctrl` records state history and transition counts so a termination policy sweep is measurable (HAS Table 3.7). The telemetry exposes:

- a per-rank rolling history of the policy state (IDLE / RD / WR)
- counters for IDLE→RD, IDLE→WR, RD→IDLE, WR→IDLE, and RD↔WR transitions
- a sticky "PARK never left" indicator for the single-rank case

12.5 FSM policy

The block permits either a small policy FSM per rank or a counter-plus-compare implementation. The states are IDLE, RD, and WR as shown in the policy table. A pure combinational decode of `grant_cmd` is not sufficient: the ODTL delays need a time reference, so every output change is launched from a loaded counter, not from the decode alone. In practice this means the FSM's next- state logic feeds a counter load, and the pin flips on counter expiration.

12.6 Timing

- ODTLon: runtime CSR, assertion delay from write command to ODT active.
- ODTLoff: runtime CSR, de-assertion delay after burst end to ODT inactive.
- Write-to-read ODT turnaround: runtime CSR, gap between write and read ODT pictures.
- tADC: runtime CSR, command-to-ODT-change interval for transitions.
- tMOD: enforced at the init seam by `init_sequencer` before `init_done` rises.
- All numeric values are runtime CSRs, initialised from the JESD79-4 speed bin at CSR-derivation time (HAS Ch 5; numeric constants are HAS open question Q1).

12.7 Notes

- The single-rank design point exercises IDLE and WR only; RD is reachable in multi-rank builds but the structure is the same.
- Rank coupling is built in, not bolted on: the “other rank terminates” rule is just the per-rank state table evaluated for every rank each cycle.
- No command ever waits for `odt_ctrl`. If a latency cannot be met because the scheduler changed its mind, the block follows the most recent grant; the scheduler’s timing rules must already guarantee that the DRAM can absorb the command.

13 Training Interfaces (andesite_wrlvl_ifc, andesite_rdlvl_ifc, andesite_ca_train_ifc)

Module: andesite_wrlvl_ifc (MODIFIED, carried and landed), andesite_rdlvl_ifc (NEW, landed), andesite_ca_train_ifc (NEW, landed) **Location:** projects/components/mem-ctrl-ip/research-ip/andesite-ddr4-lpddr4/rtl/fub/ **Category:** training control / PHY handshake **Parent:** andesite_training_plane / andesite_top **Status:** all three landed; the tables below name the landed port lists

13.1 Purpose

These three blocks expose DRAM training as controller-side sequencing engines. They handle the DFI handshake, the mode-register or MPC path, the JEDEC timing windows, and the result capture. The actual delay-line search lives in firmware — the controller provides the interface, not the algorithm. This page collects the family because all three share the same handshake discipline, the same telemetry framing, and the same no-search rule.

13.1.1 The three family rules

1. **No search loop in hardware.** Anywhere. The blocks handshake, sequence, window, and report; firmware walks the delays.
2. **Four-state telemetry.** The host must distinguish never attempted, converged, timed out, and no result in window. A detector that has never fired is not evidence of anything — that's the pumice CRC lesson, and this family applies it to training.
3. **CSR-defined timeouts with distinct status.** Where a JEDEC window's maximum is controller-dependent, the CSR defines it with a distinct status bit. A host that waits forever for a result it will never get looks the same as a broken link.

13.2 Parameters

Table 2.9.1: Training-interface parameters

Parameter	Type	Range	Default	Meaning	PRD
`NUM_CS`	int	1..8	1	number of independent chip selects (ranks or channels)	B1
`CSW`	int	—	`$clog2(NUM_CS)` (min 1)	chip-select index width (`cs_sel_i`)	B1

The JEDEC timing windows arrive as runtime-CSR ports on the landed `wrlvl_ifc` — `t_wldqsen_i`, `t_wlmrd_i`, `t_wlo_i`, `t_wloe_i` (write leveling, B2) — initialised from the JESD79-4 or JESD209-4 speed bin at CSR-derivation time (HAS Ch 5; numeric constants are HAS open question Q1). The per-interface timeout is `t_wlmrd_max_i` (B5). The rdlvl/CA-training windows (B3, B4) land with those interfaces.

13.3 Interface

13.3.1 DFI training signal groups

Signal	Direction	Width	Description
dfi_phylvl_req_cs_n_o	out	NUM_CS	controller requests per-CS write leveling (DFI leveling handshake, active-low per CS)
dfi_phylvl_ack_cs_n_i	in	NUM_CS	PHY acknowledges the training request per CS
dfi_phy_wrlvl_cs_n_o	out	NUM_CS	write-leveling chip-select to the PHY
dfi_wrlvl_strobe_o	out	1	DQS strobe the PHY forwards for the prime-DQ sample
dfi_phy_rdlvl_cs_n_o	out	NUM_CS	read-leveling chip-select, driven in rdlvl_ifc's HANDSHAKE
dfi_phylvl_req_cs_n_i	in	NUM_CS	PHY read-leveling request per CS (rdlvl_ifc handshake input)

The controller initiates the handshake — the host owns the delay sweep (design decision D2), so there is no PHY-driven request pin on wrlvl_ifc; rdlvl_ifc consumes the PHY's request and raises its own ack (dfi_phylvl_ack_cs_n_o is shared by both interfaces' handshakes on the DFI bus, one training kind at a time).

Table 2.9.2: DFI 4.0 training signals

13.3.2 CSR / host side

Signal	Direction	Width	Description
wrlvl_en_i	in	1	enter write-leveling mode (the MR1[7] path)
strobe_i	in	1	host strobe — one DQS edge per pulse; the host owns the delay sweep
cs_sel_i	in	CSW	chip select under training
t_wldqsen_i	in	16	tWLDQSEN window
t_wlmrd_i	in	16	tWLMRD window
t_wlmrd_max_i	in	16	tWLMRD timeout (0 = no timeout — the distinct timeout status)
t_wlo_i	in	16	tWLO window
t_wloe_i	in	16	tWLOE window
prime_dq_i	in	1	sampled prime DQ returned by the PHY
result_valid_o	out	1	a sample completed
result_o	out	1	captured prime-DQ value
obs_attempts_o	out	16	enable/strobe count — how many times the interface ran
obs_flips_o	out	16	result-bit transitions observed across the sweep

Signal	Direction	Width	Description
obs_timeout_o	out	1	t_wlmrd_max_i expired (distinct status)
obs_ever_done_o	out	1	converged at least once
obs_state_o	out	3	FSM-state observability
rdlvl_en_i	in	1	start one read-leveling sweep (a delay setting)
csr_mr3_mpr_enter_i	in	16	MR3 image with MPR enable + pattern select (the RTL makes no MPR-bit-position claim — HAS Q1); csr_mr3_mpr_exit_i is the disable image
t_mpr_enter_i	in	16	MRW ack to valid training pattern; t_mpr_exit_i / t_mpr_readout_i are the exit and capture windows
t_rdlvl_timeout_i	in	16	read-leveling handshake bound (0 = none)
mpr_pattern_i	in	1	observed DQ, latched per chip select in CAPTURE
cmd_req_o	out	1	MRW/MPC request path (shared shape with the init

Signal	Direction	Width	Description
			sequencer)
cmd_ack_i	in	1	formatter/ scheduler grant for the request
cmd_op_o	out	5	OP_MRS (rdlvl_ifc) / OP_MPC (ca_train_ifc)
cmd_bank_o	out	3	MR index (3 = MR3 on the rdlvl_ifc MRW path)
cmd_addr_o	out	16	the MR3 image on the MRW path
ca_train_en_i	in	1	start one CA-training round; wdq_cal_en_i starts one WDQ-calibration round
csr_mpc_ca_enter_i	in	6	MPC opcode image for CA-training entry (CSR; encodings TBC(JESD209-4) — driven, never decoded); csr_mpc_ca_exit_i, csr_mpc_wdq_enter_i, csr_mpc_wdq_exit_i are the sibling images
t_ca_train_i	in	16	CA sample window; t_wdq_cal_i is

Signal	Direction	Width	Description
			the WDQ window; `t_ca_timeout_i` bounds the MPC handshake
`ca_sample_i`	in	1	CA observation during SAMPLE; `wdq_sample_i` is the WDQ observation
`mpc_op_o`	out	6	the opcode image presented on the MPC issue path
`chan_sel_i`	in	1	LPDDR4 x16 channel select (channels train independently)

The telemetry discipline is the landed `wrlvl_*`/`obs_*` shape above, one counter set per interface.

Table 2.9.3: Host/CSR training interface

13.4 Microarchitecture internals

13.4.1 `wrlvl_ifc`: write leveling, MODIFIED

The contract is inherited from scoria unchanged. scoria's HAS Ch 3.3 / its book owns the original contract: DFI leveling handshake, MR1 path in and out of write-leveling mode, the `tWLMRD`, `tWLDQSEN`, `tWLO`, and `tWLOE` windows as runtime CSRs, prime-DQ result capture, and the four-state telemetry. The marking is MODIFIED only because the shared training machinery — handshake discipline, window enforcement, and telemetry framing — is factored so all three interfaces ride it. What `wrlvl_ifc` owes the host is exactly what it owed before.

13.4.2 `rdlvl_ifc`: MPR read leveling, NEW

DDR4's fly-by topology skews DQS against CK on the read side too. MR3's MPR (multi-purpose register) makes the DRAM drive a known training pattern onto DQ, so the controller can observe its read capture timing without the memory array in the loop. The sequence is:

```
MRW(MR3, MPR enable + pattern select)
  -> [DFI read-leveling handshake per DFI v4.0]
  -> drive MPR pattern
  -> host reads captured values
  -> MRW(MPR disable)
```

The DFI read-leveling handshake uses `dfi_phylvl_req_cs_n`, `dfi_phylvl_ack_cs_n`, and `dfi_phy_rdlvl_cs_n`, with `dfi_lvl_pattern` carrying the training pattern, per DFI v4.0 `§TBC(TASK-005)`. The MPR readout interval and its pre/post-pattern windows are runtime CSRs, named per JESD79-4, initialised from the speed bin at CSR-derivation time (HAS Ch 5; numeric constants are HAS open question Q1). Result capture is per chip select.

13.4.3 `ca_train_ifc`: LPDDR4 CA/WDQ training, NEW

LPDDR4's CA bus needs training DDR4 doesn't: LPDDR4 samples CA against CK, and both edges matter. The write-DQ path also needs its own calibration step. JESD209-4 carries both through MPC training opcodes, which is why this block sits beside `zq_ctrl`'s MPC submodule and reuses the formatter's LPDDR4 CA path. The two flows are:

```
enter CA training (MPC)
  -> sample CA against CK
  -> host reads sample
  -> adjust
  -> exit CA training (MPC)
```

and

```
enter WDQ calibration (MPC)
  -> drive/training pattern on WDQ
  -> host reads captured value
  -> adjust
  -> exit WDQ calibration (MPC)
```

State is per channel because LPDDR4's two x16 channels train independently. The training intervals are runtime CSRs, initialised from the JESD209-4 speed bin at CSR-derivation time (HAS Ch 5; numeric constants are HAS open question Q1). Whether CA training is mandatory before operation is board-dependent; the init chapter records it as a conditional step, but the interface exists either way.

13.4.4 Four-state telemetry

Every interface reports the same 2-bit status, plus three counters:

```
status encoding:
  2'b00 = never attempted
  2'b01 = converged
  2'b10 = timed out
  2'b11 = no result in window

counters (per interface):
  attempts  : how many times enable was asserted
  results   : how many times a valid capture completed
  timeouts  : how many times the timeout CSR expired before
convergence
```

A detector that has never fired is not evidence of anything. The "never attempted" state exists so firmware can tell the difference between "I haven't tried" and "I tried and failed".

13.5 FSM policy

Each interface keeps a minimal handshake FSM. For `rdlvl_ifc` the states are:

```
IDLE -> MPR_ENTRY -> HANDSHAKE -> CAPTURE -> MPR_EXIT -> REPORT ->
IDLE
```

- `IDLE`: waiting for `rdlvl_enable`.
- `MPR_ENTRY`: issue `MRW(MR3, MPR enable + pattern select)` and wait the MPR entry interval.
- `HANDSHAKE`: drive `dfi_phylvl_ack_cs_n` and `dfi_phy_rdlvl_cs_n` until the PHY acknowledges, per DFI v4.0 `§TBC(TASK-005)`.
- `CAPTURE`: latch the observed MPR pattern per chip select.
- `MPR_EXIT`: issue `MRW(MR3, MPR disable)` and wait the MPR exit interval.
- `REPORT`: update the four-state telemetry and counters, then return to `IDLE`.

There are no search states. The host walks the delay line by asserting enable again with a new delay setting. The same shape applies to `wrlvl_ifc` and `ca_train_ifc`, with the command path switched from MR3 to MR1 and from MPR to MPC respectively.

13.6 Timing

- `tWLMRD`: runtime CSR, MR1-write to DQS assertion for write leveling.
- `tWLDQSEN`: runtime CSR, DQS enable window for write leveling.
- `tWLO`: runtime CSR, DQS-to-ODT window observed during write leveling.
- `tWLOE`: runtime CSR, DQS enable-to-DQ window observed during write leveling.
- MPR readout interval: runtime CSR, MR3 entry to valid training pattern.
- MPR pre/post-pattern intervals: runtime CSRs, margins around the MPR drive.
- CA/WDQ training intervals: runtime CSRs per JESD209-4.
- `tMOD`: runtime CSR, wait after any MRW before the next valid command; used at MPR entry/exit and at write-leveling MR1 entry/exit.
- All numeric values are runtime CSRs, initialised from the JESD79-4 or JESD209-4 speed bin at CSR-derivation time (HAS Ch 5; numeric constants are HAS open question Q1).

13.7 Notes

- `wrlvl_ifc` is MODIFIED only by factoring; its visible contract is the same as scoria's.
- `rdlvl_ifc` never decides a delay. It only exposes the MPR pattern so firmware can decide.
- `ca_train_ifc` is LPDDR4-only in practice, but the structure is generic; the MPC issuer is reused with `zq_ctrl`.
- The three FUBs instantiate in `andesite_training_plane`, which owns the DFI training pins, the one-active mux, and the maintenance-class command channel (`trn_cmd_*`) into the scheduler. `init_sequencer` adds training-entry states after ZQ, `mode_register` carries MR3 MPR fields and LPDDR4 training MRs, and `dfi_cmd_formatter`'s LPDDR4 CA submodule issues the MPC opcodes. No datapath block changes for training.

14 DFI Datapath (andesite_dfi_cmd_path, andesite_dfi_rd_aligner, andesite_dfi_wr_serializer, andesite_dfi_plane)

Module: andesite_dfi_cmd_path, andesite_dfi_rd_aligner, andesite_dfi_wr_serializer, andesite_dfi_plane **Location:** projects/components/mem-ctrl-ip/research-ip/andesite-ddr4-lpddr4/rtl/ **Category:** DFI 4.0 boundary / datapath **Parent:** andesite_top / PHY interface **Status:** implemented (TASK-016 t6)

14.1 Purpose

This page covers four modules because the DFI 4.0 delta touches them as one plane. The command path widens for DDR4's extra pins; the read aligner and write serializer grow DBI handling; and the DFI plane presents the new control surface. The CDC (`dfi_cdc`) and `dfi_signal_pack` stay inherited and dormant per the HAS, so they don't get their own MAS page.

14.2 Parameters

Table 2.10.1: DFI datapath parameters

Parameter	Type	Range	Default	Meaning	PRD
NUM_RANKS	int	1..4	1	rank count for chip-select fanout	C3
NUM_BANKS	int	—	8	bank count	—
NUM_BG	int	—	4	bank-group count	—
ROW_WIDTH	int	—	14	row address width	—
COL_WIDTH	int	—	10	column address width	—
ADDR_WIDTH	int	—	18	DFI address bus width	—
DFI_RATE	int	1,2,4	2	DFI frequency ratio	—
DRAM_BEAT_WIDTH	int	PHY-defined	64	DRAM beat DQ width	C1
DFI_DATA_WIDTH	int	DRAM_BEAT_WIDTH * DFI_RATE	128	DQ width at the DFI boundary	C1
CMD_FIFO_DEPTH	int	power of 2	8	command CDC FIFO depth	—
WD_FIFO_DEPTH	int	power of 2	16	write-data CDC FIFO depth	—
RD_FIFO_DEPTH	int	power of 2	32	read-data	—

Parameter	Type	Range	Default	Meaning	PRD
				CDC FIFO depth	
RD_MAX_OUTSTANDING	int	—	16	read-aligner outstanding-read depth	—
RD_EN_CYC	int	—	4	dfi_rddata_en window width (DFI cycles)	—
BL_WORDS	int	—	RD_EN_CYC	DFI words captured per read	—

14.3 Interface

The full pin-level inventory of the DFI 4.0 boundary lives in `ch03_interfaces/01_dfi40_pins.md`; this page describes the modules behind the signals, not the signal list itself. The cross-reference is intentional — that page owns the table so this one doesn't duplicate it.

Table 2.10.2: DFI datapath signal groups

Signal group	Source block	DFI 4.0 clause suffix
Command pins (`dfi_act_n*`, `dfi_bg*`, `dfi_parity_in*`, etc.)	`dfi_cmd_path` / `dfi_cmd_formatter`	§TBC(TASK-005)
Read data + DBI	`dfi_rd_aligner`	§TBC(TASK-005)
Write data + DBI	`dfi_wr_serializer`	§TBC(TASK-005)
Training handshakes	Owned by `andesite_training_plane`, not routed here	§TBC(TASK-005)

14.4 Microarchitecture internals

14.4.1 dfi_cmd_path: widened command bus

dfi_cmd_path inherits scoria's registered pipeline and widens the command word to carry the DDR4 additions:

```
inherited: dfi_cs_n, dfi_ras_n, dfi_cas_n, dfi_we_n, dfi_address,
dfi_bank
added:     dfi_act_n, dfi_bg[1:0], dfi_parity_in
```

The added pins are driven from dfi_cmd_formatter: dfi_act_n is the fifth command pin, dfi_bg[1:0] carry the bank-group decode from addr_mapper, and dfi_parity_in carries the CA parity bit. For LPDDR4 these pins leave over the formatter's LPDDR4 CA submodule instead; dfi_cmd_path simply passes the wider container through.

14.4.2 dfi_rd_aligner: DBI on the read path

The read aligner presents dfi_rddata_dbi alongside the read data. DBI is enabled per-byte from the MR5 read-DBI image: when enabled, the DRAM inverts bytes whose DBI bit is set at send time, and the receiver (PHY/controller) restores them. The aligner reports the inversion mask to the upper planes aligned to the same beat as the data. Read DBI is MR5-programmed and independent of write DBI.

```
per-byte behavior:
  if MR5[read DBI enable] == 1:
     dfi_rddata_dbi[n] reports whether byte n was inverted
  else:
     dfi_rddata_dbi[n] is zero / don't-care
```

14.4.3 dfi_wr_serializer: DBI on the write path

The write serializer applies dfi_wrdata_dbi to the write data before it reaches the PHY. When MR5 write-DBI is enabled, each byte whose DBI bit is set is inverted on the way out; when disabled, the DBI vector is ignored. Write CRC is explicitly out of scope this edition (HAS Ch 3.1; inert in MR2) — the serializer carries no CRC lane, no CRC state machine, and no stub. It is named here so nobody wires a placeholder.

```
per-byte behavior:
  if MR5[write DBI enable] == 1:
     byte_out[n] = dfi_wrdata[n] ^ dfi_wrdata_dbi[n]
  else:
     byte_out[n] = dfi_wrdata[n]
```

14.4.4 dfi_plane: the 4.0 control surface

andesite_dfi_plane is a structural assembly in rtl/macro/:

```
andesite_dfi_cdc        : ctl_clk <-> dfi_clk async FIFOs (cmd,
wrdata, rddata)
andesite_dfi_cmd_path   : widened command word -> DFI 4.0 pin vector +
fire strobes
andesite_dfi_wr_serializer : drive dfi_wrdata/en/mask at t_phy_wrlat
andesite_dfi_rd_aligner    : capture dfi_rddata and push into the read
CDC FIFO
```

`andesite_dfi_signal_pack` stays dormant per the HAS and is not instantiated. Training pins (write-leveling, read-leveling, CA/WDQ training) live on `andesite_training_plane`, which owns the DFI training pins and their PHY clock-domain crossing — they are deliberately not carried through the DFI datapath plane.

The command word is the widened scheduler container `{ap,col,row,bg,bank,rank,op}`; the CDC command FIFO width is parameterized so the plane passes the full word without narrowing.

The single-rank design point drives the DFI v3.1 CS-qualified data lanes `dfi_wrdata_cs_o` and `dfi_rddata_cs_o` to constant zero, matching scoria's v3.1 precedent. Only `dfi_cs_o` (from `dfi_cmd_path`) is active.

`dfi_cke_o` is the registered `cke_i` input, the real CKE pin generated by the P1 init sequencer. The P1 formatter's own `dfi_cke` output is a DDR4 placeholder and is intentionally not forwarded.

`dfi_init_start_o` is the CDC's registered copy of `init_busy_i`. There is no `dfi_error` / `dfi_error_info` at this plane: scoria never had them here, and they are not added in andesite (the MAS claim is reconciled here).

14.4.5 LPDDR4 delta

The datapath structure is memory-type neutral. Only the command pin mapping changes: for LPDDR4, the command word leaves through the formatter's 6-bit CA submodule as a double-data-rate CA bus, two cycles per command. The read and write data paths are unchanged. LPDDR4's two x16 channels are a build parameter the datapath doesn't care about — per-channel state lives in `ca_train_ifc` and the refresh path, not here.

14.5 FSM policy

There are no FSMs in this plane. `dfi_cmd_path`, `dfi_rd_aligner`, and `dfi_wr_serializer` are registered pipelines and muxing, inherited shape. The DBI enable qualification is combinational on the MR5 image. `dfi_plane` itself is a routing and registration shell; the only “state” is the CDC FIFOs in the inherited `dfi_cdc`, and those keep scoria’s elastic-buffer policy.

14.6 Timing

- The command path is a registered pipeline; latency is one DFI clock per stage, matching scoria's verified shape.
- The read aligner and write serializer add no FSM latency beyond the inherited pipeline; DBI inversion is combinational on the MR5 image.
- Gear-down entry timing is owned by `init_sequencer`; `dfi_plane` only presents the handshake pins, per DFI 4.0 `§TBC(TASK-005)`.
- All DFI 4.0 clause references are suffixed `§TBC(TASK-005)` because the specification is acquired and studied under andesite TASK-005.

14.7 Notes

- The DFI 4.0 `_pN` phase notation is a generalization, not a new signal. The 1:4 frequency ratio remains a defined mode of the newer revision, per DFI 4.0 `§TBC(TASK-005)`.
- Write CRC is out of scope this edition. The datapath has no CRC lane and no stub; if the condition in HAS Ch 3.1 changes, the change starts with the MR2 image and a new datapath specification.
- The full pin table belongs to `ch03_interfaces/01_dfi40_pins.md`; this page intentionally cross-references rather than duplicates it.

15 DFI 4.0 Pin-Level Table

This page pins the boundary the HAS's Chapter 4 names: every signal HAS Table 4.1 lists appears here as a pin-level row — name, direction, clock domain, reset value, and the source block that drives or consumes it. The HAS carries the argument; this table is what the DV framework's DFI signal catalog checks against when the BFM exists (andesite TASK-005).

Citation discipline, restated from the HAS because it matters more here: the DFI 4.0 specification is on disk (`/mnt/data/github/dfi-specs/DDR_PHY_Interface_Specification_v4_0.pdf`) and was studied on 2026-10-04. Clause references below are real DFI 4.0 section numbers; any remaining `§TBC(TASK-005)` marks a claim the study could not confirm. The "behavior" column states what andesite does, not what a clause number says.

Reset convention for the whole table: DFI outputs reset to the de-asserted, command-inactive level (chip selects high, command pins high, data enables low); DFI inputs are qualified by their valid/enable companions and need no reset value. Per-signal reset conventions of DFI 4.0 itself are listed in Table 3, "DFI Signal Requirements" (§2.2.3 DFI Signals).

15.1 Command and address pins

Table 3.1: DFI 4.0 command and address pins

Pin	Dir	Clock	Reset	Source block	Behavior	Citation
`dfi_cs`	out	DFI ctrl	1	cmd_formatter	chip select; rank select from `rank_i` (v4.0 rename of `dfi_cs_n`)	§3.1 Control Interface
`dfi_act_n`	out	DFI ctrl	1	cmd_formatter	activate select; 0 marks the ACT form	§3.1 Control Interface
`dfi_ras_n`	out	DFI ctrl	1	cmd_formatter	command pin; A16 during non-ACT commands	§3.1 Control Interface
`dfi_cas_n`	out	DFI ctrl	1	cmd_formatter	command pin; A15 during non-ACT commands	§3.1 Control Interface
`dfi_we_n`	out	DFI ctrl	1	cmd_formatter	command pin;	§3.1 Control

Pin	Dir	Clock	Reset	Source block	Behavior	Citation
					A14 during non-ACT commands	Interface
dfi_bank	out	DFI ctrl	0	cmd_formatter	bank address (DDR4: BA[1:0]; LPDDR4 holds idle)	§3.1 Control Interface
dfi_bg	out	DFI ctrl	0	cmd_formatter	bank-group address BG[1:0], DDR4 only	§3.1 Control Interface
dfi_address	out	DFI ctrl	0	cmd_formatter	address bus; row on ACT, column on RD/WR, MR data on MRS	§3.1 Control Interface
dfi_cke	out	DFI ctrl	0	cmd_formatter	clock enable; ownership held by init until READY (HAS Ch 3.2 seam)	§3.1 Control Interface
dfi_rese	out	DFI ctrl	0	init_sequ	DRAM	§3.1

Pin	Dir	Clock	Reset	Source block	Behavior	Citation
t_n				encer	RESET#, driven through the init window; presented under its DFI name at the boundary (scoria convention, HAS Ch 4.1)	Control Interface
dfi_parity_in	out	DFI ctrl	0	cmd_formatter	CA parity generation point; qualified by parity_en_i	§3.5.7 CRC and CA Parity
dfi_alert_n	in	DFI ctrl	—	NEW alert handling at the boundary	parity/training alert return from PHY; logged by the formatter, recovery owned by the sequence	§3.5.7 CRC and CA Parity

Pin	Dir	Clock	Reset	Source block	Behavior	Citation
					r (HAS Ch 3.2)	
`ca_o[5:0]`	out	DFI ctrl (DDR)	0	cmd_formatter LPDDR4 CA submodule	LPDDR4 CA bus, double-data-rate, two cycles per command	§3.14 Channels for LPDDR4 memories
`ca_valid_o`	out	DFI ctrl	0	cmd_formatter LPDDR4 CA submodule	marks both CA beat cycles	andesite internal qualifier

`dfi_cid` (chip ID) is not driven at `NUM_RANKS = 1`; it is presented constant, the same disposition scoria recorded for the v3.1 data-phase chip selects (HAS Ch 4.1). A multi-rank build drives it from the granted command's rank (§3.1 Control Interface, §4.3 3DS Stack Support).

15.2 Gear-down

Table 3.2: Gear-down entry

Pin	Dir	Clock	Reset	Source block	Behavior	Citation
`dfi_geardown_en`	out	DFI ctrl	idle	init_sequencer	DDR4 geardown enable; when asserted the MC changes CA signals only every other DFI PHY clock (§4.18)	§3.13 Geardown Mode

15.3 Training and leveling pins

Table 3.3: Training and leveling pins

Pin	Dir	Clock	Reset	Source block	Behavior	Citation
`dfi_phy_wrlvl_cs`	out	DFI ctrl	1	training_plane	selects write leveling (v4.0 rename of `dfi_phy_wrlvl_cs_n`)	§3.6 DFI Training Interface
`dfi_phy_rdlvl_cs`	out	DFI ctrl	1	training_plane	selects read leveling; unused by scoria, driven by `rdlvl_ifc` (v4.0 rename of `dfi_phy_rdlvl_cs_n`)	§3.6.1 Read Training Operation
`dfi_phy_rdlvl_gate_cs`	out	DFI ctrl	1	training_plane	read-gate leveling select; presented, exercised per BFM capability (HAS Q3) (v4.0 rename	§3.6.1 Read Training Operation

Pin	Dir	Clock	Reset	Source block	Behavior	Citation
					of `dfi_phy_rdlvl_gate_cs_n`)	
`dfi_phy_calvl_cs`	out	DFI ctrl	1	training_plane	selects CA training; driven by `ca_train_ifc` (v4.0 rename of `dfi_phy_calvl_cs_n`)	§3.6.4 CA Training Operation
`dfi_lvl_pattern`	out	DFI ctrl	0	training_plane	training pattern select	§3.6.6 dfi_lvl_pattern Encoding
`dfi_lvl_periodic`	out	DFI ctrl	0	training_plane	periodic-training control	§3.6.7 Periodic Training Flag
`dfi_calvl_en`	out	DFI ctrl	0	training_plane	MC enables CA training logic	§3.6.4 CA Training Operation
`dfi_calvl_req`	in	DFI ctrl	—	training_plane	PHY requests CA training	§3.6.4 CA Training Operation
`dfi_calvl_capture`	out	DFI ctrl	0	training_plane	CA-training capture	§3.6.4 CA Training Operatio

Pin	Dir	Clock	Reset	Source block	Behavior	Citation
					strobe from MC to PHY	n
dfi_calvl_resp	in	DFI ctrl	—	training_plane	CA-training response from PHY	§3.6.4 CA Training Operation
dfi_calvl_ca_sel	out	DFI ctrl	0	training_plane	CA foreground pattern strobe	§3.6.4 CA Training Operation
dfi_calvl_data	out	DFI data phases	0	training_plane	LPDDR4 CA VREF training data (per slice)	§3.6.4 CA Training Operation
dfi_calvl_done	out	DFI data phases	0	training_plane	LPDDR4 CA VREF training done	§3.6.4 CA Training Operation
dfi_calvl_result	in	DFI data phases	—	training_plane	LPDDR4 CA VREF training result from PHY	§3.6.4 CA Training Operation
dfi_calvl_strobe	out	DFI data phases	0	training_plane	LPDDR4 CA VREF training strobe	§3.6.4 CA Training Operation

15.4 Data-path pins (DBI and the inherited surface)

Table 3.4: Data-path and inherited pins

Pin	Dir	Clock	Reset	Source block	Behavior	Citation
`dfi_wrdata` / `dfi_wrdata_mask`	out	DFI data phases	0	dfi_datapath	write data and mask/DBI; `dfi_wrdata_mask` becomes write DBI when DBI is enabled and `phydbi_mode = 0`; phase-multiplied `_pN`, inherited	§3.2.1 Write Data Mask/Write DBI
`dfi_wrdata_en` / `dfi_wrdata_cs`	out	DFI data phases	0	dfi_datapath	write data enable; per-phase chip select driven constant at one rank (scoria convention; v4.0	§3.2.2 Write Data Chip Select

Pin	Dir	Clock	Reset	Source block	Behavior	Citation
					rename of `dfi_wrdata_cs_n`)	
`dfi_rddata` / `dfi_rddata_valid`	in	DFI data phases	—	dfi_datapath	read data and valid; inherited capture	§3.3.5 Read Data Signals and Parameters
`dfi_rddata_dbi`	in	DFI data phases	—	dfi_datapath	read DBI, per byte; presented alongside `dfi_rddata`	§3.3.1 Read DBI
`dfi_rddata_cs`	out	DFI data phases	1	dfi_datapath	per-data-phase chip select, constant at one rank (inherited; v4.0 rename of `dfi_rddata_cs_n`)	§3.3.2 Read Data Chip Select
`dfi_error` / `dfi_error_info`	in	DFI ctrl	—	boundary (alert handling)	error indication and info; scoria's	§3.8 Error Interface

Pin	Dir	Clock	Reset	Source block	Behavior	Citation
					surface, DDR4 CA-parity detail now used (HAS Ch 4.1)	
`dfi_init_start`	out	DFI ctrl	active	init_sequencer via the DFI plane	MC init/frequency-change request; generated at the DFI plane from the P1 init FSM's busy/done (the P1 sequencer is self-timed and emits no PHY handshake of its own)	§3.5.1 Initialization
`dfi_init_complete`	in	DFI ctrl	—	boundary	PHY init complete / frequency-change acknowle	§3.5.1 Initialization

Pin	Dir	Clock	Reset	Source block	Behavior	Citation
					dge; driven by the PHY, not consumed at this design point (self-timed init)	
`dfi_lp_ctrl_req` / `dfi_lp_data_req`	out	DFI ctrl	0	dormant (`powerdown_ctrl` pair)	exposed per the timeout-and-report rule; nothing consumes them at this design point (HAS Ch 3.1)	§3.7 Low Power Control Interface

15.5 Study corrections (2026-10-04, andesite TASK-005)

- **Active-low chip-select names are v3.1, not v4.0.** DFI 4.0 renames `dfi_cs_n` → `dfi_cs`, `dfi_wrdata_cs_n` → `dfi_wrdata_cs`, `dfi_rddata_cs_n` → `dfi_rddata_cs`, and all training chip selects from `dfi_phy_*_cs_n` to `dfi_phy_*_cs` (§3.1, §3.2.2, §3.3.2, §3.6). The original table used v3.1 names; this edition uses v4.0 names.
- **`dfi_phylvl_req_cs_n` / `dfi_phylvl_ack_cs_n` are not DFI 4.0 signals.** They were defined in v3.1 as PHY-requested training handshakes and are superseded in v4.0 by PHY Independent Mode (§3.6.8). They have been removed from this v4.0 pin table.
- **`dfi_wrdata_dbi` does not exist.** Write DBI reuses `dfi_wrdata_mask` when DBI is enabled and `phydbi_mode = 0` (§3.2.1). The separate `dfi_wrdata_dbi` pin has been removed.
- **`dfi_ca_capture` was wrong in name and direction.** The DFI 4.0 signal is `dfi_calvl_capture`, it is MC-to-PHY (out), and the PHY-to-MC observation returns are `dfi_calvl_resp` and `dfi_calvl_result` (§3.6.4).
- **`dfi_init` is not a DFI signal name.** The spec defines `dfi_init_start` (MC-to-PHY) and `dfi_init_complete` (PHY-to-MC) (§3.5.1). Both are now listed separately.

15.6 What this table does not do

It does not restate the inherited datapath's phase structure (the `_pN` buses and the 1:4 ratio are argued in HAS Ch 4.1 and owned by dfi_datapath). DFI 4.0 clause numbers are now cited above; any remaining `§TBC(TASK-005)` marks a claim the study could not confirm.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

16 Core Signal Contracts

16.1 What a contract is here

A signal contract in this repo is a machine-checkable statement of what a signal is allowed to do: a term list with its defining `file:line` citation, the invariants between terms, and a decision table whose forbidden rows are marked `ILLEGAL`. The canonical methodology is `vault/handbook/design/signal-contracts-and-kmaps.md`; the shared generator machinery is `bin/kmaps/`, and the andesite kmap book (`../kmaps/`, andesite TASK-004) builds its contract sheets from the anchors this chapter names.

16.2 Pre-RTL citation posture

At MAS v0.1 there is no RTL, so every contract below cites the MAS ch02 page where the intended expression is written verbatim — the fenced anchors. When the first RTL lands, each citation is re-pointed at the corresponding `.sv` file and line, and the generator's citation gate then enforces that the workbook and the RTL agree; a drift fails the run. The anchor map:

Table 4.1: Anchor map — the kmap generator cites these file:line pairs

Contract domain	Defining anchors (file:line, at v0.1)
DDR4 command decode	01_cmd_formatter.md — truth table fence
Activate field packing	01_cmd_formatter.md — activate form fence
LPDDR4 CA encodings	01_cmd_formatter.md — placeholder table, pinned by the kmap book
Init FSM ordering	02_init_sequencer.md — DDR4 and LPDDR4 state-list fences
Address decode structure	04_addr_mapper.md — decode fence
Scheduler admission	05_scheduler.md — admission-rule fence
FGR interval arithmetic	06_refresh_ctrl.md — interval fence
MPC calibration FSM	07_zq_ctrl.md — MPC FSM fence
ODT policy states	08_odt_ctrl.md — policy-state fence
Training flows and telemetry	09_training.md — MPR/CA/WDQ flow fences, four-state encoding fence
DFI datapath pin set and DBI	10_dfi_datapath.md — pin-set and DBI fences
LPDDR4 CA encodings	generated/02_lpddr4_ca_command_table.md — TBC JESD209-4
Address decode	generated/03_addr_decode_maps.md — design-point decode maps
MR programming	generated/04_mr_programming_maps.md — MR0-MR6 / LPDDR4 MRW

Contract domain	Defining anchors (file:line, at v0.1)
ODT truth	generated/05_odt_truth_table.md — termination policy
FGR select	generated/06_fgr_refresh_map.md — refresh granularity

The line numbers are captured at authoring time by the generator's citation gate; the table above names the fences, and `verify_citations` pins the exact lines on every run.

16.3 Contract: DDR4 command decode (the formatter's core)

Terms. `ACT_n`, `RAS_n`, `CAS_n`, `WE_n` (DFI command pins, source 01_cmd_formatter.md); `CS_n` (select); `AP` (the auto-precharge address bit, A10).

Invariants (each renders ILLEGAL rows impossible, per the truth-table anchor):

1. Exactly one of the anchored encodings holds whenever `CS_n = 0`; the fence's rows are exhaustive and mutually exclusive.
2. `ACT_n = 0` marks exactly three rows in the anchor: ACT (with `RAS_n=1, CAS_n=1, WE_n=1`), MRS (`RAS_n=0, CAS_n=0, WE_n=0`) and REF (`RAS_n=0, CAS_n=0, WE_n=1`) — no other pin combination may carry `ACT_n = 0`.
3. `AP` is an address input: it never appears in the pin-level table; the variants (RDA/WRA, PREA, ZQCL) are the RD/WR/PRE/ZQ rows qualified by `AP = 1`.
4. Parity (`dfi_parity_in`) is generated only when `parity_en_i = 1`, and `parity_en_i` may only be set after the init sequence has programmed parity mode (HAS Ch 3.2 dependency).

Decision table. One row per (command, AP) combination the controller issues; forbidden rows — commands outside the anchored set, or any command while `parity_en_i` disagrees with the MR image — are `ILLEGAL`. The minimized qualifier SOPs for this table are the kmap book's DDR4 command table (andesite TASK-004 output); the workbook and this contract must always cite the same anchor lines.

16.4 Contract: maintenance request/grant

Terms. `req`/`gnt` between `refresh_ctrl`, `zq_ctrl` (both cores), `odt_ctrl` turnaround notices, and the scheduler (source 05_scheduler.md).

Invariants.

1. A maintenance source raises `req` and waits; it never asserts its command onto the bus without `gnt` (family doctrine 2, quoted in full in the family docs).
2. `gnt` is exclusive: at most one maintenance source holds the bus in any cycle.
3. A refresh of either granularity and a ZQ calibration of either form are ordinary occupants of the banks they touch: the bank timers own those banks for the operation's duration (HAS Ch 3.4 requirement 4).

Decision table. One row per (source, state) pair; rows where two sources hold `gnt`, or a source issues without `gnt`, are `ILLEGAL`.

16.5 Contract: training four-state telemetry

Terms. The 2-bit status and counters per interface (source 09_training.md).

Invariants.

1. The status is exactly one of {never attempted, converged, timed out, no result in window}; a detector that has never fired is not evidence of anything, so “never attempted” must be representable and is the reset state.
2. A timeout is distinguishable from a no-result: the CSR-defined timeout with its distinct status bit is the family’s rule (HAS Ch 3.3), and the two states may not share an encoding.
3. No interface state performs a search: the state lists contain no delay-walk states, and the host-visible counters only count attempts, results, and timeouts.

Decision table. One row per (interface, status) pair; any row pairing a search behaviour with a legal status is `ILLEGAL`.

16.6 Regenerating the workbook

```
cd projects/components/mem-ctrl-ip/research-ip/andesite-ddr4-lpddr4
python3 docs/kmaps/gen_andesite_kmaps.py
```

The generator writes `docs/kmaps/andesite_cmd_kmaps.xlsx` and the `docs/kmaps/generated/*.md` renderings. Do not hand-edit the `.xlsx`; it is a build artifact of the generator, and the citation gate fails the run if any `file:line` citation drifts from this book.

For every contract the verdict is `NOT CHECKED` until RTL exists — the correct pre-RTL posture, identical to the bch MAS's. The verdict becomes meaningful once `rtl_sop` can be diffed against the derived minimal cover.